UNIVERSIDAD DE SEVILLA

FACULTAD DE MATEMÁTICAS

Trabajo de Fin de Máster

Uso de datos en formato grafos en algoritmos de aprendizaje automático (Machine Learning) supervisados

Autor: Luis García Marín

Tutor: David Gálvez Ruiz

Departamento: Estadística e Investigación Operativa

Resumen

Este trabajo estudia las redes neuronales sobre grafos como extensión de las redes neuronales clásicas y convolucionales a datos con estructura relacional. Se presentan los fundamentos de los grafos y del *message passing*, así como varias arquitecturas GNN: GCN, GraphSAGE, GIN, GINE, GAT, GATv2 y GGNN. La parte experimental se centra en la clasificación de grafos moleculares del dataset MUTAG, evaluando los modelos bajo un protocolo común con validación cruzada, varias semillas, validación interna y *early stopping*. Los resultados muestran que las arquitecturas con intercambio de información entre nodos superan al baseline MLP+pool, y que el modelo GINE+GATv2 obtiene el mejor rendimiento medio. Finalmente, se incluye un análisis de explicabilidad mediante GNNExplainer para estudiar qué subestructuras moleculares influyen en las predicciones.

Abstract

This work studies graph neural networks as an extension of classical and convolutional neural networks to relational data. It introduces the foundations of graphs and message passing, together with several GNN architectures: GCN, GraphSAGE, GIN, GINE, GAT, GATv2 and GGNN. The experimental section focuses on molecular graph classification using the MUTAG dataset, evaluating all models under a common protocol with cross-validation, multiple random seeds, internal validation and early stopping. The results show that message passing architectures outperform the MLP+pool baseline, and that the GINE+GATv2 model achieves the best average performance. Finally, an explainability analysis based on GNNExplainer is included to study which molecular substructures influence the predictions.

Índice

1. Introducción **4**

2. Aprendizaje estadístico **6**

3. Redes neuronales clásicas **8**
3.1. El perceptrón 8
3.1.1. Regla de aprendizaje 9
3.1.2. Teorema de convergencia 9
3.2. Perceptrón multicapa (MLP): construcción, no linealidad y entrenamiento 10
3.2.1. Aproximación universal. 13
3.3. Retropropagación 15

4. Convoluciones en dominios regulares **19**
4.1. Redes neuronales convolucionales (CNN) 19
4.2. Equivarianza y pesos compartidos 21

5. Fundamentos de grafos **22**
5.1. Grafos, matrices y Laplaciano 23
5.2. Señales sobre grafos 26
5.3. Permutaciones, invariancia y equivarianza 27
5.4. Tareas de aprendizaje sobre grafos 29

6. De CNN a GNN **30**
6.1. La imagen como caso particular de dominio regular 30
6.2. La dificultad de extender un kernel convolucional a grafos 31
6.3. Enfoque espectral: filtros definidos mediante el Laplaciano 32
6.4. Enfoque espacial: agregación local y message passing 33

7. Arquitectura GNN **34**
7.1. Graph Convolutional Network (GCN) 34
7.2. GraphSAGE 35
7.3. Graph Isomorphism Network (GIN) 36
7.3.1. Contexto: agregación de vecindarios, multiconjuntos y poder expresivo 36
7.3.2. Relación con Weisfeiler–Lehman (1-WL) y un criterio de optimalidad 37
7.3.3. Suma como agregador universal de multiconjuntos 38
7.3.4. Definición de la capa GIN 38
7.3.5. Readout a nivel de grafo y clasificación 39
7.3.6. Extensión natural cuando hay atributos en aristas: GINE 39
7.3.7. Correspondencia explícita con la interfaz `MessagePassing` en PyTorch Geometric 40
7.3.8. Motivación y planteamiento 40

7.3.9. Capa de atención sobre grafos (GAT) 40
7.3.10. Interpretación como Message Passing 41
7.3.11. Invarianza a permutaciones en el vecindario 41
7.3.12. Atención multi-cabeza 42
7.3.13. Complejidad computacional 42
7.3.14. Regularización y detalles prácticos 42
7.3.15. Limitación expresiva: atención estática en GAT 43
7.3.16. GATv2: una corrección para obtener atención dinámica . 44
7.3.17. Extensión a atributos de arista (comentario) 44
7.3.18. Lectura (readout) para clasificación a nivel de grafo . . . 44
7.4. Gated Graph Neural Network (GGNN) 45
7.4.1. Planteamiento: clasificación de grafos en el paradigma $f(G(N(x), V(x))) = y_{\text{class}}$ 45
7.4.2. Estados ocultos y padding inicial 45
7.4.3. Message Passing recurrente: mensajes y agregación 45
7.4.4. Actualización con compuertas: GRU sobre grafos 46
7.4.5. Forma matricial 46
7.4.6. Relación con otros paradigmas: sistema dinámico y MPNN 47
7.4.7. Readout para clasificación de grafos 47
7.4.8. Correspondencia con `GatedGraphConv` de PyTorch Geometric 47

8. Experimentación **48**
8.1. Dataset: MUTAG 48
8.2. Protocolo de experimentación 53
8.3. Resultados 55
8.4. Explicabilidad de las predicciones mediante GNNExplainer . . . 60
8.4.1. Protocolo de explicabilidad 60
8.4.2. Ejemplo de explicación en una molécula mutagénica correctamente clasificada 61
8.4.3. Comparación entre aciertos y errores 62
8.4.4. Patrones agregados observados 63
8.4.5. Limitaciones del análisis XAI 64

9. Conclusiones **64**

1. Introducción

El aprendizaje automático se ha consolidado como una herramienta fundamental para construir modelos capaces de extraer patrones a partir de datos y utilizarlos para realizar predicciones sobre nuevas observaciones. Desde el punto de vista de la teoría del aprendizaje estadístico, este problema puede formularse como la elección de una función dentro de una determinada clase de hipótesis, a partir de una muestra finita, con el objetivo de que el comportamiento aprendido no se limite a los datos de entrenamiento, sino que generalice adecuadamente a datos no observados [1, 2]. Esta tensión entre ajuste y generalización atraviesa buena parte del desarrollo moderno del aprendizaje automático.

Dentro del conjunto de algoritmos de aprendizaje automático localizamos las redes neuronales, una familia especialmente flexible de modelos parametrizados. Su interés reside tanto en su capacidad para aproximar funciones complejas como en la posibilidad de adaptar su arquitectura a la estructura del problema considerado. En particular, el paso desde modelos lineales simples, como el perceptrón, hasta redes multicapa con funciones de activación no lineales permite construir representaciones internas cada vez más ricas de los datos [3, 5]. Los resultados clásicos de aproximación universal muestran que, bajo hipótesis adecuadas, las redes feedforward poseen una gran capacidad representacional [10, 11, 12]; sin embargo, en aplicaciones reales no basta con disponer de una clase de funciones expresiva. También es necesario incorporar en el modelo información previa sobre la naturaleza de los datos.

Un ejemplo paradigmático de esta idea son las redes neuronales convolucionales. En lugar de tratar una imagen como un vector arbitrario de píxeles, las CNN aprovechan que las imágenes están definidas sobre una rejilla regular. La conectividad local, la compartición de pesos y la equivarianza a traslaciones permiten construir modelos más eficientes y mejor alineados con la estructura espacial de la entrada [5]. De este modo, el diseño de la arquitectura introduce un sesgo inductivo adecuado: los mismos patrones visuales pueden aparecer en distintas posiciones de la imagen, y por ello tiene sentido reutilizar los mismos filtros en todo el dominio.

No obstante, muchos datos de interés no poseen una estructura euclídea regular. En numerosos problemas, la información relevante no está organizada sobre una línea, una cuadrícula o un volumen, sino mediante relaciones entre entidades. Las moléculas pueden representarse como átomos unidos por enlaces, las redes sociales como usuarios conectados por interacciones, los sistemas de recomendación como relaciones entre usuarios y productos, y ciertas infraestructuras como redes de nodos y conexiones. En todos estos casos, el grafo proporciona una representación natural: los nodos describen entidades y las aristas codifican relaciones entre ellas [13, 15].

Esta diferencia estructural plantea una dificultad esencial. En una imagen, el vecindario de un píxel tiene un orden geométrico claro: existe una noción de arriba, abajo, izquierda, derecha o diagonal. En un grafo arbitrario, en cambio, los vecinos de un nodo no poseen un orden canónico y su número puede variar de un nodo a otro. Por tanto, no es posible trasladar de forma directa la noción

clásica de kernel convolucional a un grafo general. Cualquier operación neuronal sobre grafos debe respetar que el etiquetado de los nodos es arbitrario: si se reordenan los nodos, la información representada por el grafo no cambia. Esta observación conduce a la necesidad de arquitecturas invariantes o equivariantes frente a permutaciones [15].

Las redes neuronales sobre grafos, o *Graph Neural Networks* (GNN), surgen como respuesta a este problema. Su principio básico consiste en actualizar la representación de cada nodo mediante la información procedente de su vecindario, utilizando operaciones de agregación que no dependan del orden en que se enumeran los vecinos. Esta idea se formaliza en el paradigma de *message passing*, donde los nodos intercambian mensajes a través de las aristas, agregan la información recibida y actualizan sus estados internos [16, 17]. Al apilar varias capas, cada nodo puede incorporar información de regiones progresivamente más amplias del grafo.

La construcción de modelos neuronales sobre grafos puede entenderse desde dos perspectivas complementarias. Por un lado, el enfoque espectral utiliza el Laplaciano del grafo y sus autovectores para definir una noción de filtrado análoga a la transformada de Fourier clásica [6, 14]. Por otro lado, el enfoque espacial describe directamente operaciones locales de agregación en los vecindarios, lo que conecta de forma natural con el paso de mensajes y con muchas de las arquitecturas modernas [16]. Ambas perspectivas permiten interpretar las GNN como una generalización de las ideas que hicieron efectivas a las CNN, pero adaptadas a dominios irregulares.

En este trabajo se estudian los fundamentos matemáticos y computacionales que permiten pasar desde las redes neuronales clásicas y convolucionales hasta las redes neuronales sobre grafos. El interés principal no es únicamente presentar una colección de arquitecturas, sino comprender qué propiedades estructurales justifican su diseño: localidad, compartición de parámetros, agregación de información de vecindarios, invariancia a permutaciones y capacidad para combinar atributos de nodos y aristas. En este marco se analizan modelos representativos como GCN, GraphSAGE, GIN, GAT, GATv2 y GGNN, que proponen distintas formas de propagar, ponderar o actualizar la información en el grafo [18, 47, 21, 33, 34, 39].

La parte experimental se centra en una tarea de clasificación de grafos moleculares utilizando el dataset MUTAG, un conjunto clásico incluido en TUDataset [29]. En este problema, cada molécula se representa como un grafo cuyos nodos corresponden a átomos y cuyas aristas representan enlaces químicos. La etiqueta se asigna a la molécula completa, por lo que el modelo debe transformar información local, asociada a átomos y enlaces, en una predicción global. Esta tarea constituye un escenario adecuado para estudiar el comportamiento de distintas arquitecturas GNN y valorar cómo sus mecanismos de agregación y lectura influyen en la clasificación final.

El objetivo general del trabajo es, por tanto, ofrecer una visión coherente de las redes neuronales sobre grafos desde sus fundamentos hasta su aplicación práctica. Para ello se parte del marco general del aprendizaje estadístico, se revisan las ideas esenciales de las redes neuronales y convolucionales, y se introduce

posteriormente el lenguaje de grafos necesario para formular arquitecturas capaces de operar sobre dominios relacionales. La experimentación con MUTAG, implementada mediante PyTorch Geometric [42], permite conectar la teoría con un problema concreto de clasificación molecular y observar cómo las distintas decisiones arquitectónicas se reflejan en el rendimiento empírico.

2. Aprendizaje estadístico

La teoría del aprendizaje estadístico surge a finales de los años 60 como un marco esencialmente teórico para analizar el problema de *estimar funciones* a partir de una colección finita de datos. Durante décadas, su contribución principal fue proporcionar herramientas para entender, con precisión matemática, cuándo y por qué un procedimiento de ajuste puede *generalizar* más allá de la muestra. A mediados de los años 90, la aparición de algoritmos como las máquinas de vectores de soporte (*support vector machines*,SVM), construidos explícitamente a partir de estos principios, consolidó la teoría no sólo como instrumento de análisis sino también como guía para diseñar métodos prácticos de estimación de funciones multivariantes de gran dimensionalidad [1].

El aprendizaje estadístico puede formularse del siguiente modo [2]. Sea $\mathcal{X}$ un espacio de entradas y $\mathcal{Y}$ un espacio de salidas. Se supone que los pares $(X, Y) \in \mathcal{X} \times \mathcal{Y}$ están distribuidos según una distribución desconocida P. Dado que P no es accesible directamente, sólo se dispone de una muestra finita

$$S = \{(x_i, y_i)\}_{i=1}^{n},$$

formada por observaciones independientes e idénticamente distribuidas según P.

Se fija una clase de hipótesis $\mathcal{H}$, cuyos elementos son funciones candidatas $h : \mathcal{X} \to \mathcal{Y}$, y una función de pérdida

$$\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_+,$$

que mide el coste de predecir $h(x)$ cuando la respuesta verdadera es y. El objetivo ideal consiste en encontrar una hipótesis que minimice el *riesgo poblacional*

$$R(h) = \mathbb{E}_{(X,Y)\sim P}\big[\ell(h(X), Y)\big].$$

El minimizador ideal sería

$$h^\star \in \arg\min_{h\in\mathcal{H}} R(h).$$

Sin embargo, como la distribución P es desconocida, el riesgo poblacional no puede evaluarse directamente. En su lugar se minimiza el *riesgo empírico*

$$\widehat{R}_S(h) = \frac{1}{n}\sum_{i=1}^{n} \ell(h(x_i), y_i),$$

obteniendo una hipótesis

$$\widehat{h}_S \in \arg\min_{h\in\mathcal{H}} \widehat{R}_S(h).$$

El problema fundamental del aprendizaje estadístico consiste en que la hipótesis seleccionada a partir de la muestra no sólo tenga bajo error empírico, sino también bajo riesgo poblacional. Esta capacidad de comportarse bien sobre observaciones no vistas se denomina *generalización* [1]

La diferencia entre el comportamiento en la muestra y fuera de ella puede expresarse mediante el *gap de generalización*

$$R(\widehat{h}_S) - \widehat{R}_S(\widehat{h}_S).$$

En general, minimizar de forma excesiva el riesgo empírico puede conducir a sobreajuste, especialmente cuando la clase de hipótesis es muy expresiva en relación con el tamaño de la muestra. Por ello, en la práctica se introduce con frecuencia un término de regularización[2], dando lugar a problemas de la forma

$$\widehat{h}_S \in \arg\min_{h\in\mathcal{H}} \left[\widehat{R}_S(h) + \lambda\Omega(h)\right],$$

donde $\Omega(h)$ penaliza la complejidad de la hipótesis y $\lambda \geq 0$ controla la intensidad de dicha penalización.

En el caso supervisado con respuesta cuantitativa, una formulación ampliamente utilizada, propuesta en [4], asume que observamos una variable respuesta Y y p predictores $X_1, \ldots, X_p$, con $X = (X_1, \ldots, X_p)$, y que existe una relación

$$Y = f(X) + \varepsilon,$$

donde f es una función fija pero desconocida y ε es un término aleatorio independiente de X con media cero. En esta formulación, f representa la parte sistemática de la información que X aporta sobre Y, mientras que ε recoge la variabilidad no explicada.

La hipótesis de independencia entre ε y X, junto con $\mathbb{E}[\varepsilon] = 0$, es en bastante reestrictiva, pudiendose trabajar con condiciones más laxas, como:

$$\mathbb{E}[\varepsilon \mid X] = 0.$$

Bajo esta condición se tiene

$$\mathbb{E}[Y \mid X] = \mathbb{E}[f(X) + \varepsilon \mid X] = f(X) + \mathbb{E}[\varepsilon \mid X] = f(X).$$

Por tanto, $f(X)$ puede interpretarse como la esperanza condicional de la respuesta dada la entrada, mientras que ε recoge la desviación respecto de dicha componente sistemática.

Desde esta perspectiva, los métodos de aprendizaje automático pueden entenderse como procedimientos para elegir una función dentro de una familia determinada. En particular, las redes neuronales definen clases parametrizadas de funciones

$$\mathcal{H}_\Theta = \{f_\theta : \mathcal{X} \to \mathcal{Y} \mid \theta \in \Theta\},$$

donde θ representa el conjunto de parámetros entrenables. Entrenar una red neuronal consiste entonces en resolver, de forma exacta o aproximada, un problema de minimización empírica de la forma

$$\widehat{\theta} \in \arg\min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i),$$

posiblemente acompañado de regularización.

3. Redes neuronales clásicas

3.1. El perceptrón

El perceptrón de Rosenblatt [3] es un clasificador binario lineal. Para una entrada $\mathbf{x} \in \mathbb{R}^m$, un vector de pesos $\mathbf{w} \in \mathbb{R}^m$ y un sesgo $b \in \mathbb{R}$, se define el campo local inducido

$$v = \mathbf{w}^T \mathbf{x} + b. \tag{1}$$

La salida se obtiene mediante un limitador duro:

$$y = \operatorname{sign}(v) = \begin{cases} +1, & v \geq 0, \\ -1, & v < 0. \end{cases} \tag{2}$$

Por tanto, la frontera de decisión está determinada por el hiperplano

$$\mathbf{w}^T \mathbf{x} + b = 0, \tag{3}$$

que divide $\mathbb{R}^m$ en los semiespacios

$$\mathbf{w}^T \mathbf{x} + b \geq 0, \qquad \mathbf{w}^T \mathbf{x} + b < 0.$$

El vector $\mathbf{w}$ fija la orientación del hiperplano y el sesgo b determina su desplazamiento respecto del origen.

Para simplificar la notación, se incorpora el sesgo como una coordenada adicional. Se define

$$\tilde{\mathbf{x}} = \begin{bmatrix} 1 \\ \mathbf{x} \end{bmatrix} \in \mathbb{R}^{m+1}, \qquad \tilde{\mathbf{w}} = \begin{bmatrix} b \\ \mathbf{w} \end{bmatrix} \in \mathbb{R}^{m+1}. \tag{4}$$

Entonces

$$v = \tilde{\mathbf{w}}^T \tilde{\mathbf{x}}, \qquad y = \operatorname{sign}\big(\tilde{\mathbf{w}}^T \tilde{\mathbf{x}}\big). \tag{5}$$

Sea

$$\mathcal{S} = \{(\tilde{\mathbf{x}}_i, d_i)\}_{i=1}^{N}, \qquad d_i \in \{-1, +1\},$$

un conjunto de entrenamiento. El patrón $\tilde{\mathbf{x}}_i$ está correctamente clasificado si

$$d_i \, \tilde{\mathbf{w}}^T \tilde{\mathbf{x}}_i > 0. \tag{6}$$

El objetivo del perceptrón es encontrar un vector $\tilde{\mathbf{w}}$ que satisfaga esta desigualdad para todos los patrones del conjunto de entrenamiento.

3.1.1. Regla de aprendizaje

El algoritmo del perceptrón actualiza los pesos únicamente cuando el patrón presentado está mal clasificado. Dado un patrón $(\tilde{\mathbf{x}}_i, d_i)$, se calcula

$$y_i = \operatorname{sign}\big(\tilde{\mathbf{w}}^T \tilde{\mathbf{x}}_i\big).$$

Si $y_i = d_i$, no se modifica el vector de pesos. Si $y_i \neq d_i$, se aplica la actualización

$$\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + \eta\, d_i\, \tilde{\mathbf{x}}_i, \qquad \eta > 0. \tag{7}$$

De forma equivalente, usando la salida $y_i \in \{-1, +1\}$, puede escribirse

$$\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + \frac{\eta}{2}(d_i - y_i)\tilde{\mathbf{x}}_i. \tag{8}$$

En efecto, si la clasificación es correcta entonces $d_i - y_i = 0$; si es incorrecta, $d_i - y_i = 2d_i$, y (8) coincide con (7).

El algoritmo puede resumirse como sigue:

1. Inicializar $\tilde{\mathbf{w}} = \mathbf{0}$ o con valores pequeños.
2. Presentar sucesivamente los patrones $(\tilde{\mathbf{x}}_i, d_i)$.
3. Calcular $y_i = \operatorname{sign}(\tilde{\mathbf{w}}^T \tilde{\mathbf{x}}_i)$.
4. Si $y_i \neq d_i$, actualizar $\tilde{\mathbf{w}}$ mediante (7).
5. Repetir el proceso hasta que no se produzcan errores o hasta alcanzar un criterio de parada.

La propiedad esencial para el análisis de convergencia es que toda actualización se realiza sobre un patrón mal clasificado. Por tanto, en cada actualización se cumple

$$d_i\, \tilde{\mathbf{w}}^T \tilde{\mathbf{x}}_i \leq 0. \tag{9}$$

3.1.2. Teorema de convergencia

Siguiendo la demostración propuesta en [7], sea $\mathcal{S} = \{(\tilde{\mathbf{x}}_i, d_i)\}_{i=1}^{N}$, con $d_i \in \{-1, +1\}$. Supóngase que el conjunto es linealmente separable, es decir, que existe un vector $\mathbf{u} \in \mathbb{R}^{m+1}$, $\mathbf{u} \neq \mathbf{0}$, y una constante $\alpha > 0$ tales que

$$d_i\, \mathbf{u}^\top \tilde{\mathbf{x}}_i \geq \alpha, \qquad i = 1, \ldots, N. \tag{10}$$

Supóngase además que los patrones están acotados:

$$\|\tilde{\mathbf{x}}_i\|_2^2 \leq \beta, \qquad i = 1, \ldots, N, \tag{11}$$

para alguna constante $\beta > 0$.

Entonces, para $\eta = 1$ e inicialización $\tilde{\mathbf{w}}^{(0)} = \mathbf{0}$, el algoritmo del perceptrón realiza un número finito M de actualizaciones. Además,

$$M \leq \frac{\beta \|\mathbf{u}\|_2^2}{\alpha^2}. \tag{12}$$

Demostración. Denotemos por $\tilde{\mathbf{w}}^{(k)}$ el vector de pesos después de la k-ésima actualización. En la actualización k, el algoritmo usa algún patrón mal clasificado $(\tilde{\mathbf{x}}_k, d_k)$, de modo que

$$\tilde{\mathbf{w}}^{(k)} = \tilde{\mathbf{w}}^{(k-1)} + d_k \tilde{\mathbf{x}}_k. \tag{13}$$

Primero se acota inferiormente la proyección sobre un separador:

$$\mathbf{u}^\top \tilde{\mathbf{w}}^{(k)} = \mathbf{u}^\top \tilde{\mathbf{w}}^{(k-1)} + d_k \mathbf{u}^\top \tilde{\mathbf{x}}_k \geq \mathbf{u}^\top \tilde{\mathbf{w}}^{(k-1)} + \alpha.$$

Por inducción,

$$\mathbf{u}^\top \tilde{\mathbf{w}}^{(M)} \geq M\alpha. \tag{14}$$

Por otro lado,

$$\begin{aligned} \|\tilde{\mathbf{w}}^{(k)}\|_2^2 &= \|\tilde{\mathbf{w}}^{(k-1)} + d_k \tilde{\mathbf{x}}_k\|_2^2 \\ &= \|\tilde{\mathbf{w}}^{(k-1)}\|_2^2 + 2d_k (\tilde{\mathbf{w}}^{(k-1)})^\top \tilde{\mathbf{x}}_k + \|\tilde{\mathbf{x}}_k\|_2^2. \end{aligned}$$

Como la actualización se produce solo cuando el patrón está mal clasificado,

$$d_k (\tilde{\mathbf{w}}^{(k-1)})^\top \tilde{\mathbf{x}}_k \leq 0.$$

Usando (11),

$$\|\tilde{\mathbf{w}}^{(k)}\|_2^2 \leq \|\tilde{\mathbf{w}}^{(k-1)}\|_2^2 + \beta.$$

Iterando desde $\tilde{\mathbf{w}}^{(0)} = \mathbf{0}$,

$$\|\tilde{\mathbf{w}}^{(M)}\|_2^2 \leq M\beta, \qquad \|\tilde{\mathbf{w}}^{(M)}\|_2 \leq \sqrt{M\beta}. \tag{15}$$

Por Cauchy-Schwarz,

$$\mathbf{u}^\top \tilde{\mathbf{w}}^{(M)} \leq \|\mathbf{u}\|_2 \, \|\tilde{\mathbf{w}}^{(M)}\|_2 \leq \|\mathbf{u}\|_2 \sqrt{\beta M}.$$

Combinando esta desigualdad con (14),

$$M\alpha \leq \|\mathbf{u}\|_2 \sqrt{M\beta}.$$

Si $M > 0$, dividiendo por $\sqrt{M}$ y elevando al cuadrado se obtiene

$$M \leq \frac{\beta \|\mathbf{u}\|_2^2}{\alpha^2}.$$

Por tanto, el número de actualizaciones es finito. □

3.2. Perceptrón multicapa (MLP): construcción, no linealidad y entrenamiento

Para mantener la notación introducida en la sección de aprendizaje estadístico, consideremos una muestra supervisada

$$S = \{(x_i, y_i)\}_{i=1}^n, \qquad x_i \in \mathbb{R}^p,$$

donde n denota el tamaño muestral y p la dimensión de entrada. En esta subsección se usará y_i para la respuesta observada, mientras que las salidas internas de las capas se denotarán por $a^{(s)}$. De este modo se evita confundir las etiquetas del problema de aprendizaje con las activaciones de una neurona concreta.

El perceptrón de una sola capa implementa una regla de decisión basada en un hiperplano en el espacio de entrada. Para una entrada $x \in \mathbb{R}^p$, pesos $w \in \mathbb{R}^p$ y sesgo $b \in \mathbb{R}$, el campo local inducido es

$$v = w^\top x + b.$$

Antes de aplicar la función signo, el modelo define por tanto una función afín. Esta observación muestra una limitación fundamental: si únicamente se combinan unidades afines mediante sumas lineales, la familia resultante sigue siendo afín.

Proposición. Sean

$$\psi_j(x) = u_j^\top x + c_j, \qquad j = 1, \dots, r,$$

unidades afines sobre $\mathbb{R}^p$, con $u_j \in \mathbb{R}^p$ y $c_j \in \mathbb{R}$. Entonces cualquier combinación lineal

$$F(x) = \beta_0 + \sum_{j=1}^{r} \beta_j \psi_j(x)$$

es de nuevo una función afín.

Demostración. Sustituyendo la expresión de cada ψ_j, se obtiene

$$F(x) = \beta_0 + \sum_{j=1}^{r} \beta_j (u_j^\top x + c_j) = \left(\sum_{j=1}^{r} \beta_j u_j \right)^\top x + \left(\beta_0 + \sum_{j=1}^{r} \beta_j c_j \right).$$

Por tanto,

$$F(x) = \widetilde{u}^\top x + \widetilde{c},$$

donde

$$\widetilde{u} = \sum_{j=1}^{r} \beta_j u_j, \qquad \widetilde{c} = \beta_0 + \sum_{j=1}^{r} \beta_j c_j.$$

Luego F es afín. □

Así, aumentar el número de perceptrones lineales no basta para superar la separabilidad lineal: la frontera de decisión sigue estando determinada por funciones afines. La forma natural de aumentar la capacidad expresiva consiste en introducir transformaciones no lineales entre capas. Esto conduce al perceptrón multicapa, o *multilayer perceptron* (MLP), una red *feedforward* compuesta por transformaciones afines alternadas con funciones de activación no lineales.

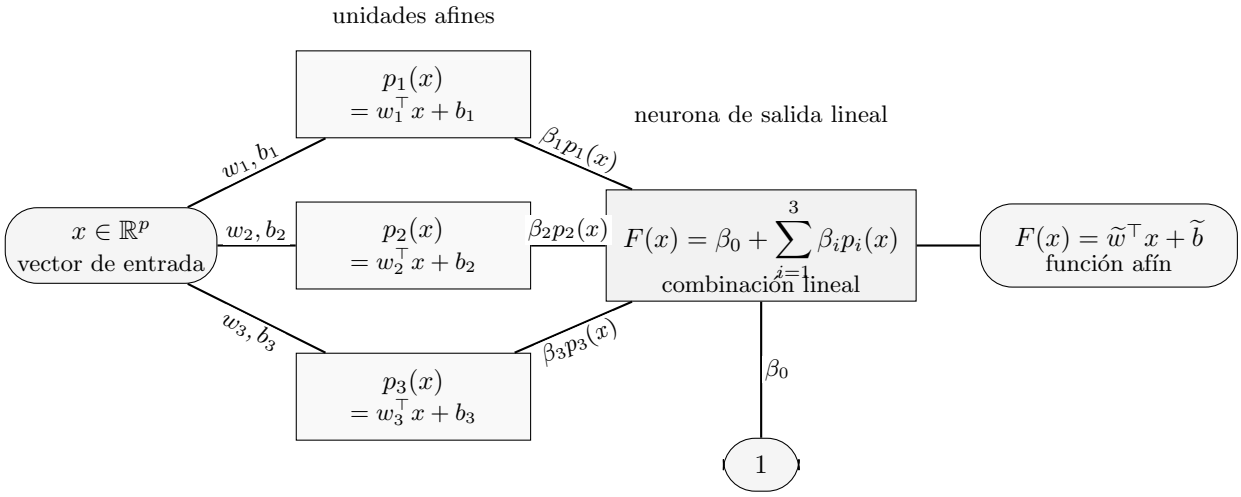

Figura 1: Combinación lineal de perceptrones lineales. Para $r = 3$, la salida sigue siendo una función afín de la entrada. Elaboración propia

Definición 3.1. *Sea $q_0 = p$. Diremos que un MLP con $L-1$ capas ocultas y una capa de salida se define recursivamente mediante*

$$a^{(0)} = x,$$

y, para $s = 1, \ldots, L-1$,

$$z^{(s)} = W^{(s)} a^{(s-1)} + b^{(s)}, \qquad a^{(s)} = \rho_s\Big(z^{(s)}\Big),$$

donde

$$W^{(s)} \in \mathbb{R}^{q_s \times q_{s-1}}, \qquad b^{(s)} \in \mathbb{R}^{q_s}.$$

La función ρ_s actúa componente a componente y se toma no lineal, normalmente continua o diferenciable casi en todo punto. Ejemplos de estas funciones serían la sigmoide, tangente hiperbólica o ReLU.

La capa de salida se escribe como

$$z_\theta(x) = W^{(L)} a^{(L-1)} + b^{(L)},$$

con

$$W^{(L)} \in \mathbb{R}^{q_L \times q_{L-1}}, \qquad b^{(L)} \in \mathbb{R}^{q_L}.$$

El conjunto de parámetros entrenables es

$$\theta = \{W^{(s)}, b^{(s)}\}_{s=1}^{L}.$$

En regresión se suele tomar directamente

$$f_\theta(x) = z_\theta(x).$$

En clasificación multiclase con C clases, se toma $q_L = C$ y se interpretan los componentes de $z_\theta(x)$ como *logits*. La salida probabilística se obtiene mediante la función softmax:

$$q_\theta(c \mid x) = \frac{\exp(z_{\theta,c}(x))}{\sum_{r=1}^{C} \exp(z_{\theta,r}(x))}, \qquad c = 1, \dots, C.$$

El clasificador inducido es entonces

$$h_\theta(x) = \arg \max_{c \in \{1,\dots,C\}} q_\theta(c \mid x).$$

Esta construcción encaja con el marco general del aprendizaje estadístico. Si

$$\mathcal{H}_\Theta = \{f_\theta : \theta \in \Theta\}$$

denota la clase de hipótesis definida por la arquitectura, entrenar el MLP consiste en resolver, de forma exacta o aproximada,

$$\widehat{\theta}_S \in \arg\min_{\theta \in \Theta} \left[\widehat{R}_S(\theta) + \lambda \Omega(\theta) \right],$$

donde

$$\widehat{R}_S(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i),$$

$\Omega(\theta)$ es un posible término de regularización y $\lambda \geq 0$ controla su intensidad. En regresión es habitual usar el error cuadrático medio, mientras que en clasificación multiclase se usa frecuentemente la entropía cruzada,

$$\ell(q_\theta(\cdot \mid x_i), y_i) = -\log q_\theta(y_i \mid x_i).$$

3.2.1. Aproximación universal.

La observación anterior justifica la necesidad de no linealidades, pero no dice todavía qué tan expresiva puede llegar a ser la clase de funciones definida por un MLP. Los primeros resultados clásicos de aproximación universal muestran que redes feedforward con una sola capa oculta y anchura arbitraria pueden aproximar uniformemente funciones continuas sobre compactos, bajo hipótesis adecuadas sobre la función de activación. En esta línea, Cybenko obtuvo el resultado fundacional para activaciones sigmoidales, mientras que Hornik dio poco después una formulación más general para activaciones continuas, acotadas y no constantes.

Definición 3.2. *Diremos que una función $\sigma : \mathbb{R} \to \mathbb{R}$ es sigmoidal si*

$$\lim_{t \to -\infty} \sigma(t) = 0, \qquad \lim_{t \to +\infty} \sigma(t) = 1.$$

Para una activación escalar $\sigma : \mathbb{R} \to \mathbb{R}$, consideremos la familia de redes de una capa oculta y salida escalar

$$\mathcal{A}_{\sigma,p} = \left\{ x \mapsto \sum_{j=1}^{r} \alpha_j \sigma(u_j^\top x + \tau_j) : r \in \mathbb{N},\ \alpha_j, \tau_j \in \mathbb{R},\ u_j \in \mathbb{R}^p \right\}.$$

Teorema 3.1 (Teorema de Cybenko)**.** *Sea σ una función continua y sigmoidal. Entonces, para toda función continua $g \in C([0,1]^p)$ y todo $\varepsilon > 0$, existe $f \in \mathcal{A}_{\sigma,p}$, es decir, existen $r \in \mathbb{N}$, coeficientes $\alpha_j \in \mathbb{R}$, pesos $u_j \in \mathbb{R}^p$ y sesgos $\tau_j \in \mathbb{R}$, tales que*

$$\sup_{x\in[0,1]^p} |g(x) - f(x)| = \sup_{x\in[0,1]^p} \left| g(x) - \sum_{j=1}^{r} \alpha_j \sigma(u_j^\top x + \tau_j) \right| < \varepsilon.$$

Es decir, las redes con una sola capa oculta, anchura arbitraria y activación sigmoidal continua son densas en $C([0,1]^p)$ con la norma uniforme [10].

El teorema de Cybenko establece así que la no linealidad no sólo evita el colapso afín, sino que permite aproximar funciones continuas arbitrarias sobre el hipercubo unidad mediante combinaciones finitas de neuronas sigmoidales. Una extensión clásica de esta idea fue dada por Hornik en 1991, quien formuló la aproximación universal bajo hipótesis menos específicas sobre la activación.

Teorema 3.2 (Teorema de Aproximación Universal.)**.** *Sea $\sigma : \mathbb{R} \to \mathbb{R}$ una función continua, acotada y no constante. Entonces, para todo compacto $K \subset \mathbb{R}^p$, el conjunto de restricciones de redes*

$$\{f|_K : f \in \mathcal{A}_{\sigma,p}\}$$

es denso en $C(K)$ con la norma uniforme, es decir,

$$\overline{\{f|_K : f \in \mathcal{A}_{\sigma,p}\}}^{\|\cdot\|_\infty} = C(K).$$

Equivalentemente, para toda función continua $g \in C(K)$ y todo $\varepsilon > 0$, existe $f \in \mathcal{A}_{\sigma,p}$ tal que

$$\sup_{x\in K} |g(x) - f(x)| < \varepsilon.$$

Por tanto, no es necesario restringirse a activaciones sigmoidales concretas: basta con que la activación sea continua, acotada y no constante para obtener densidad uniforme sobre compactos [11].

Los resultados de Cybenko y Hornik constituyen la forma clásica del teorema de aproximación universal para redes feedforward con una capa oculta. Sin embargo, estas formulaciones no cubren directamente activaciones modernas no acotadas, como ReLU. Para incluir este tipo de activaciones conviene recurrir a un resultado posterior y más general, dado por Leshno, Lin, Pinkus y Schocken, que caracteriza cuándo una activación permite aproximación universal.

Definición 3.3. *Diremos que una activación $\sigma : \mathbb{R} \to \mathbb{R}$ es admisible si es localmente acotada y continua a trozos.*

Esta definición cubre muchas de las activaciones usadas en redes neuronales actuales, como sigmoide, tangente hiperbólica y ReLU.

Teorema 3.3 (Teorema de Leshno–Lin–Pinkus–Schocken.)**.** *Sea σ una activación admisible. Entonces, para todo compacto $K \subset \mathbb{R}^p$,*

$$\overline{\{f|_K : f \in \mathcal{A}_{\sigma,p}\}}^{\|\cdot\|_\infty} = C(K)$$

si y sólo si σ no coincide casi en todas partes con un polinomio algebraico [12].

En otras palabras, para toda función continua $g \in C(K)$ y todo $\varepsilon > 0$, existe una red de una capa oculta de la forma

$$x \mapsto \sum_{j=1}^{r} \alpha_j \sigma(u_j^\top x + \tau_j)$$

tal que

$$\sup_{x \in K} \left| g(x) - \sum_{j=1}^{r} \alpha_j \sigma(u_j^\top x + \tau_j) \right| < \varepsilon$$

si y sólo si la activación no es polinómica casi en todas partes.

El teorema de Leshno–Lin–Pinkus–Schocken subsume, bajo sus hipótesis de regularidad, los casos anteriores de activaciones sigmoidales o continuas acotadas no constantes, y además justifica el uso de activaciones no polinómicas no acotadas como ReLU desde el punto de vista de la aproximación universal. El término de sesgo τ_j es esencial en esta formulación, pues permite desplazar los hiperplanos $u_j^\top x + \tau_j = 0$ y aumentar la flexibilidad de la familia de funciones. Estos resultados son teoremas de existencia y de capacidad representacional: garantizan que, si la red tiene anchura suficiente, existen parámetros capaces de aproximar una función continua dada sobre un compacto. Sin embargo, no garantizan que el algoritmo de entrenamiento encuentre necesariamente esos parámetros, ni aseguran por sí solos una buena generalización fuera de la muestra.

Por tanto, la justificación matemática del MLP no debe entenderse como un análogo directo del teorema de convergencia del perceptrón. En el perceptrón lineal separable existe una garantía algorítmica clara bajo hipótesis de separabilidad. En cambio, en un MLP general la pérdida suele ser no convexa en los parámetros, de modo que la retropropagación proporciona un método eficiente para calcular gradientes, pero no una garantía general de convergencia global. La solidez del MLP proviene de la combinación de tres elementos: su capacidad de aproximación, su formulación dentro del marco de minimización del riesgo empírico y el control práctico o teórico de la complejidad de la clase de hipótesis.

3.3. Retropropagación

La retropropagación (*back-propagation*) es el procedimiento utilizado para calcular de forma eficiente las derivadas del error respecto a los pesos sinápticos de un perceptrón multicapa. Al igual que el algoritmo LMS, modifica los pesos en la dirección opuesta al gradiente del error, aplicando a cada peso sináptico $w_{ji}(n)$

una corrección $\Delta w_{ji}(n)$ proporcional a la derivada parcial del error respecto a dicho peso. En esta subsección se seguirá la notación propuesta en [3].

Consideremos una neurona de salida j. Sea n el índice de iteración o de presentación del patrón de entrenamiento. La señal inducida local de la neurona j se define como

$$v_j(n) = \sum_{i=0}^{m} w_{ji}(n) y_i(n),$$

donde $y_i(n)$ es la señal que llega a la sinapsis i de la neurona j. El término $i = 0$ representa el sesgo, incorporado mediante la entrada fija

$$y_0(n) = +1, \qquad w_{j0}(n) = b_j(n).$$

La salida de la neurona se obtiene aplicando una función de activación diferenciable $\varphi_j(\cdot)$:

$$y_j(n) = \varphi_j(v_j(n)).$$

Para una neurona de salida j, se define la señal de error como

$$e_j(n) = d_j(n) - y_j(n),$$

donde $d_j(n)$ es la respuesta deseada e $y_j(n)$ es la respuesta producida por la red hasta dicha salida. Si C denota el conjunto de neuronas de salida, el error instantáneo total se define como

$$e(n) = \frac{1}{2} \sum_{j \in C} e_j^2(n).$$

La corrección aplicada al peso $w_{ji}(n)$ se define mediante descenso por gradiente:

$$\Delta w_{ji}(n) = -\eta \frac{\partial e(n)}{\partial w_{ji}(n)},$$

donde $\eta > 0$ es la tasa de aprendizaje. Por tanto, el problema consiste en calcular la derivada parcial

$$\frac{\partial e(n)}{\partial w_{ji}(n)}.$$

Aplicando la regla de la cadena, esta derivada puede descomponerse como

$$\frac{\partial e(n)}{\partial w_{ji}(n)} = \frac{\partial e(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \frac{\partial v_j(n)}{\partial w_{ji}(n)}.$$

Cada uno de estos factores tiene una interpretación directa. En efecto,

$$\frac{\partial e(n)}{\partial e_j(n)} = e_j(n),$$

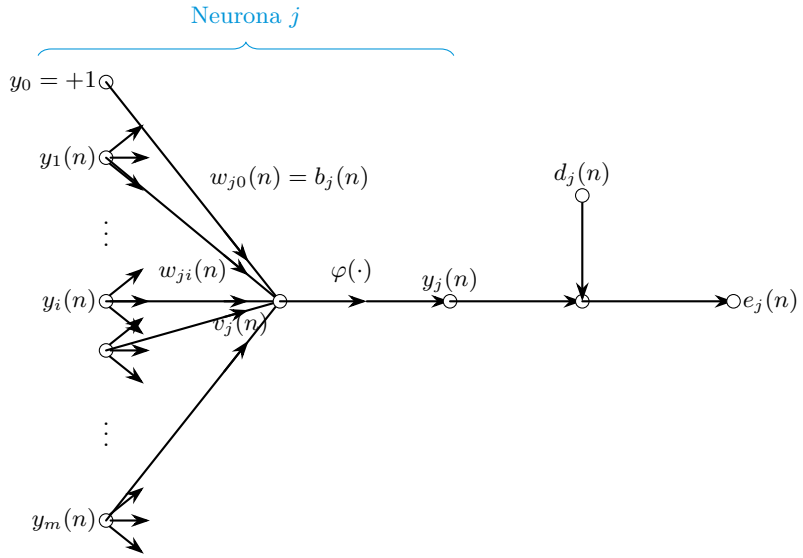

Figura 2: Grafo de flujo de señales que destaca los detalles de la neurona de salida j, adaptado de [3].

porque $e(n) = \frac{1}{2}\sum_{j\in C} e_j^2(n)$. Además, como

$$e_j(n) = d_j(n) - y_j(n),$$

se tiene

$$\frac{\partial e_j(n)}{\partial y_j(n)} = -1.$$

Por otra parte,

$$\frac{\partial y_j(n)}{\partial v_j(n)} = \varphi_j'(v_j(n)),$$

y, dado que

$$v_j(n) = \sum_{i=0}^{m} w_{ji}(n) y_i(n),$$

se obtiene

$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = y_i(n).$$

Sustituyendo estos cuatro factores en la regla de la cadena,

$$\frac{\partial e(n)}{\partial w_{ji}(n)} = -e_j(n)\varphi'_j(v_j(n))y_i(n).$$

Haykin introduce entonces el *gradiente local* de la neurona j, definido como

$$\delta_j(n) = -\frac{\partial e(n)}{\partial v_j(n)}.$$

En el caso de una neurona de salida, usando la expresión anterior, se obtiene

$$\delta_j(n) = e_j(n)\varphi'_j(v_j(n)).$$

Por tanto, la corrección del peso puede escribirse como

$$\Delta w_{ji}(n) = \eta\delta_j(n)y_i(n).$$

Esta expresión muestra que el cambio aplicado al peso $w_{ji}(n)$ es el producto de tres factores: la tasa de aprendizaje η, el gradiente local $\delta_j(n)$ de la neurona postsináptica y la señal de entrada $y_i(n)$ que llega por dicha sinapsis.

Para una neurona oculta j, no existe una respuesta deseada directa $d_j(n)$, por lo que no puede definirse una señal de error $e_j(n)$ de la misma forma que en una neurona de salida. En este caso, el gradiente local se obtiene propagando hacia atrás los gradientes locales de las neuronas de la capa siguiente. Si k recorre las neuronas a las que la neurona oculta j envía su salida, entonces

$$\delta_j(n) = \varphi'_j(v_j(n)) \sum_k \delta_k(n)w_{kj}(n).$$

Esta ecuación es el núcleo de la retropropagación: el error se calcula primero en la capa de salida y después se transmite hacia atrás, capa por capa, permitiendo asignar a cada peso una parte de responsabilidad en el error total. Una vez calculado $\delta_j(n)$, tanto para neuronas de salida como para neuronas ocultas, la actualización conserva la misma forma general:

$$\Delta w_{ji}(n) = \eta\delta_j(n)y_i(n).$$

Finalmente, el peso se actualiza mediante

$$w_{ji}(n+1) = w_{ji}(n) + \Delta w_{ji}(n).$$

Así, la retropropagación proporciona un método sistemático para calcular los gradientes locales de todas las neuronas de la red mediante la regla de la cadena y actualizar los pesos sinápticos de forma eficiente [3].

4. Convoluciones en dominios regulares

4.1. Redes neuronales convolucionales (CNN)

Cuando la entrada posee estructura espacial (imágenes, señales temporales discretizadas, rejillas), un MLP totalmente conectado ignora la localidad geométrica y fuerza a aprender desde datos invariancias que podrían incorporarse en la propia arquitectura. Las CNN introducen dos ideas estructurales: *conectividad local* (campos receptivos) y *compartición de pesos* (*weight sharing*), lo que reduce drásticamente el número de parámetros y favorece la generalización [3, 5]. En términos algebraicos, esta construcción hace que la operación lineal básica adopte la forma de una suma de convolución.

Definición 4.1 (Entrada como mapa de características multicanal)**.** *Una imagen en color se representa como un tensor* $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$*, donde* C_{in} *es el número de canales (por ejemplo* 3 *para RGB) y* H, W *son dimensiones espaciales. Cada canal* $X_{c,:,:}$ *es un mapa bidimensional. [4]*

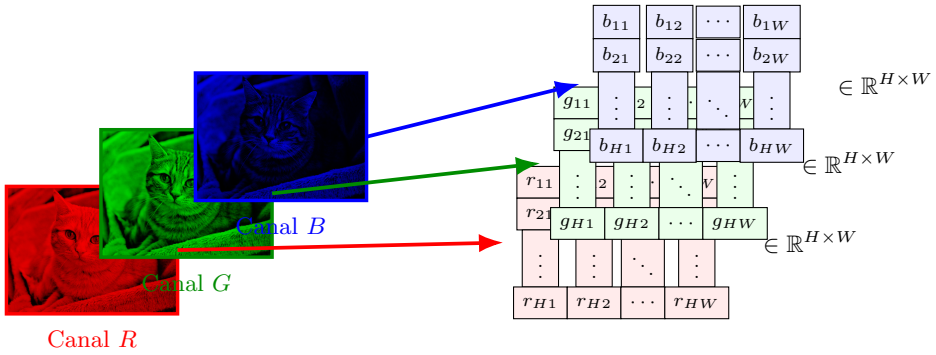

Figura 3: Descomposición de una imagen RGB en tres canales. Cada canal puede verse como una matriz $H \times W$, y la imagen completa como un tensor $X \in \mathbb{R}^{3 \times H \times W}$. Elaboración propia.

Definición 4.2 (Capa convolucional (2D) con C_{out} filtros)**.** *Sea un banco de filtros* $K \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k_H \times k_W}$ *y sesgos* $b \in \mathbb{R}^{C_{\text{out}}}$*. Definimos, para cada canal de salida* r *y posición espacial* (u, v)*,*

$$Z_{r,u,v} = \sum_{c=1}^{C_{\text{in}}} \sum_{i=0}^{k_H-1} \sum_{j=0}^{k_W-1} K_{r,c,i,j}\; X_{c,\,u+i-p,\,v+j-p} \;+\; b_r, \qquad A_{r,u,v} = \rho(Z_{r,u,v}),$$

donde p *es un parámetro de padding (se asume extensión por ceros cuando los índices quedan fuera de rango) y* ρ *es una no linealidad como ReLU. En práctica de* deep learning *esta operación se implementa como correlación cruzada, pero se denomina por tradición "convolución". [4]*

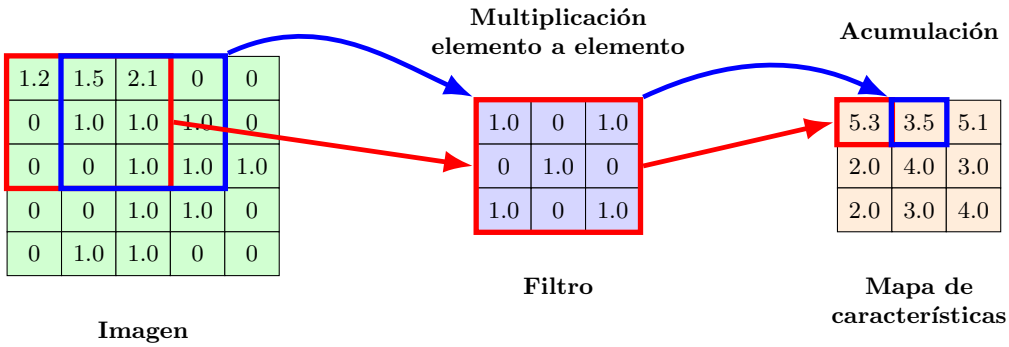

Figura 4: Ejemplo de una operación convolucional bidimensional. El recuadro rojo muestra el primer campo receptivo y produce el valor 5,3 del mapa de características. Al desplazar el mismo filtro una posición hacia la derecha, el recuadro azul produce el valor 3,5. Adaptada desde [45]

Esta definición captura explícitamente el papel de los *canales*: cada filtro $K_{r,\cdot,\cdot,\cdot}$ combina todos los canales de entrada, suma sus contribuciones y produce un nuevo mapa bidimensional (un canal de salida). Usando C_{out} filtros se obtienen C_{out} canales nuevos, formando un nuevo tensor $A \in \mathbb{R}^{C_{\text{out}} \times H' \times W'}$ [4]. Una ventaja cuantitativa clave es el conteo de parámetros: una capa convolucional tiene

$$\#\text{params} = C_{\text{out}}\big(C_{\text{in}} k_H k_W + 1\big),$$

mientras que una capa totalmente conectada sobre una imagen aplanada puede requerir millones de pesos. Esta reducción se debe al *weight sharing*: el mismo núcleo $K_{r,c,i,j}$ se aplica en todas las posiciones espaciales [3].

Observación 4.1 (Campos receptivos y suma de convolución). La conectividad local equivale a restringir, para cada neurona de la capa, qué píxeles pueden influir en su salida (su campo receptivo). Si además imponemos que distintas neuronas "desplazadas" compartan exactamente los mismos pesos, el potencial inducido se escribe como una suma de la forma $\sum_i w_i x_{i+\text{desplazamiento}}$, que es precisamente una suma de convolución discreta [3].

Pooling y reducción de resolución. Además de alternar convolución y no linealidad, es habitual insertar capas de *pooling* para condensar mapas y ganar invariancia a pequeñas traslaciones. Un ejemplo estándar es el *max pooling* 2×2:

$$(\text{Pool}(A))_{r,u,v} = \max\{A_{r,2u+i,2v+j} : i,j \in \{0,1\}\}.$$

Este operador reduce H y W a la mitad y aporta invariancia aproximada a la localización: basta con que exista una activación grande dentro del bloque para que el bloque reducido también sea grande [4].

4.2. Equivarianza y pesos compartidos

La compartición de pesos hace que, antes de operaciones de submuestreo como el *pooling*, la convolución discreta sea equivariante a traslaciones en dominios infinitos, periódicos o cuando las condiciones de contorno son compatibles con la traslación [5, 15]: si desplazamos la entrada, la salida se desplaza igual. Sea T_τ el operador de traslación discreta, definido por

$$(T_\tau X)_{u,v} = X_{u-\tau_1, v-\tau_2}.$$

En ausencia de efectos de borde, un filtro K satisface

$$K * (T_\tau X) = T_\tau (K * X). \tag{16}$$

donde $*$ denota la operación lineal de la capa convolucional (sin no linealidad).

Demostración. Para una posición (u, v), la operación convolucional discreta se escribe como

$$(K * X)_{u,v} = \sum_{\Delta u, \Delta v} K_{\Delta u, \Delta v}\, X_{u+\Delta u,\, v+\Delta v}.$$

Entonces, usando la definición de traslación

$$(T_\tau X)_{u,v} = X_{u-\tau_1,\, v-\tau_2},$$

se obtiene

$$\begin{aligned}
(K * (T_\tau X))_{u,v} &= \sum_{\Delta u, \Delta v} K_{\Delta u, \Delta v}\, (T_\tau X)_{u+\Delta u,\, v+\Delta v} \\
&= \sum_{\Delta u, \Delta v} K_{\Delta u, \Delta v}\, X_{u+\Delta u-\tau_1,\, v+\Delta v-\tau_2} \\
&= \sum_{\Delta u, \Delta v} K_{\Delta u, \Delta v}\, X_{(u-\tau_1)+\Delta u,\, (v-\tau_2)+\Delta v} \\
&= (K * X)_{u-\tau_1,\, v-\tau_2} \\
&= (T_\tau (K * X))_{u,v}.
\end{aligned}$$

Como la igualdad se verifica para toda posición (u, v), se concluye que

$$K * (T_\tau X) = T_\tau (K * X).$$

□

Esta equivarianza explica por qué un detector aprendido en una región de la imagen se reutiliza en cualquier otra, alineando la arquitectura con el conocimiento previo "los patrones pueden aparecer en cualquier posición" [3, 4].

En imágenes finitas con *zero padding*, la igualdad anterior no es exacta en los bordes: al desplazar la imagen, cambian los valores artificiales que entran en

las ventanas de convolución cercanas al contorno. Por ello, en CNN aplicadas a imágenes finitas la equivarianza debe entenderse como exacta en el interior bajo condiciones ideales, y aproximada o dependiente del tratamiento de bordes en la práctica.

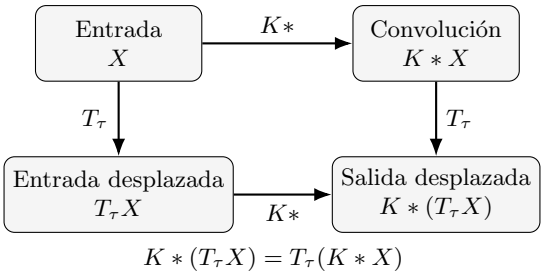

Figura 5: Equivarianza de la convolución respecto a traslaciones. Desplazar la entrada y después aplicar el filtro produce el mismo resultado que aplicar primero la convolución y desplazar después la salida.

Una CNN profunda repite bloques *convolución* → *activación* → *pooling* varias veces, incrementando el número de canales mientras disminuye la resolución espacial, hasta que los mapas resultan pequeños; entonces se "aplanan" (*flatten*) y se conectan a una o varias capas densas (un MLP final) que produce la predicción de clase [4]. Este patrón aparece tanto en descripciones modernas (mapas multicanal y pooling) como en formulaciones clásicas basadas en capas de convolución y submuestreo, donde la alternancia produce un efecto "bipiramidal": más mapas pero menor resolución [3]. Un ejemplo prototípico para reconocimiento de caracteres combina varias etapas de convolución y subsampling logrando muchas conexiones efectivas pero pocos parámetros libres gracias al *weight sharing* [3].

La construcción anterior depende de una propiedad esencial de las imágenes: los píxeles viven sobre una rejilla regular. Por ello, el vecindario de una posición (u, v) puede describirse mediante desplazamientos fijos $(\Delta u, \Delta v)$, y el mismo filtro puede aplicarse en todas las posiciones. Esta estructura desaparece en un grafo general: los vecinos de un nodo no están ordenados por direcciones espaciales fijas y el número de vecinos puede variar de un nodo a otro. Esta observación motiva la necesidad de introducir el lenguaje de grafos antes de definir una noción de convolución sobre dominios irregulares.

5. Fundamentos de grafos

En la sección anterior se ha visto que las CNN explotan la estructura regular de una rejilla mediante conectividad local, pesos compartidos y equivarianza a traslaciones. Para extender esta idea a dominios no regulares, necesitamos introducir el lenguaje matemático de los grafos. La finalidad de esta sección no

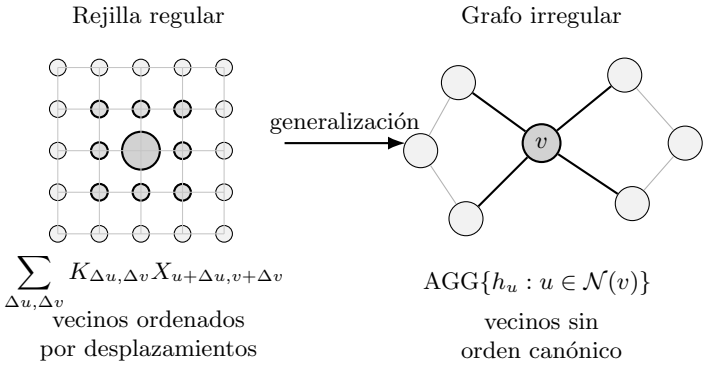

Figura 6: Diferencia entre una convolución sobre una rejilla regular y una operación local sobre un grafo. En una rejilla, los vecinos de una posición se identifican mediante desplazamientos espaciales fijos; en un grafo, el vecindario $\mathcal{N}(v)$ no posee un orden canónico, por lo que la operación debe basarse en agregadores invariantes a permutaciones. Elaboración propia.

es desarrollar la teoría de grafos en toda su generalidad, sino fijar los objetos necesarios para entender la transición de CNN a GNN: la estructura de vecindad, las matrices asociadas al grafo, las señales definidas sobre nodos y la simetría por permutación de los etiquetados.

5.1. Grafos, matrices y Laplaciano

Un grafo puede entenderse como una estructura formada por objetos y relaciones. Formalmente, consideraremos un grafo finito

$$G = (V, E),$$

donde $V = \{v_1, \ldots, v_n\}$ es el conjunto de vértices o nodos y E es el conjunto de aristas. En un grafo no dirigido, cada arista une dos vértices sin orientación, de modo que una arista puede escribirse como $\{v_i, v_j\}$. En un grafo dirigido, en cambio, las aristas tienen orientación y se escriben como pares ordenados (v_i, v_j). En este último caso se distingue entre vecinos entrantes y salientes [13].

En lo que sigue, salvo que se indique lo contrario, trabajaremos con grafos simples no dirigidos: no se permiten bucles ni aristas múltiples. Esta restricción simplifica la exposición inicial, aunque en aplicaciones de GNN es habitual extender el modelo a grafos dirigidos, ponderados, con autolazos o con atributos en aristas.

El vecindario abierto de un nodo v_i se define como

$$\mathcal{N}(v_i) = \{v_j \in V : \{v_i, v_j\} \in E\}.$$

Es decir, $\mathcal{N}(v_i)$ contiene los nodos directamente conectados con v_i. El grado de

v_i es el número de vecinos de v_i:

$$d_i = |\mathcal{N}(v_i)|.$$

En grafos ponderados, si la arista $\{v_i, v_j\}$ tiene peso $w_{ij} \geq 0$, el grado ponderado se define como

$$d_i = \sum_{j=1}^{n} w_{ij}.$$

Una forma algebraica de representar la estructura del grafo es mediante su matriz de adyacencia. Para un grafo no dirigido y no ponderado, la matriz de adyacencia $A \in \{0,1\}^{n\times n}$ se define por

$$A_{ij} = \begin{cases} 1, & \text{si } \{v_i, v_j\} \in E, \\ 0, & \text{en caso contrario.} \end{cases}$$

Si el grafo es no dirigido, entonces A es simétrica. Si el grafo es ponderado, se toma $A_{ij} = w_{ij}$. La matriz de grados se define como la matriz diagonal

$$D = \operatorname{diag}(d_1, \ldots, d_n).$$

A partir de A y D se define el Laplaciano combinatorio del grafo:

$$L = D - A.$$

Proposición 5.1 (Propiedades básicas del Laplaciano combinatorio)**.** *Sea G un grafo no dirigido con pesos no negativos y matriz de Laplaciano combinatorio $L = D - A$. Entonces:*

1. *L tiene suma nula por filas, es decir,*

$$\sum_{j=1}^{n} L_{ij} = 0, \qquad i = 1, \ldots, n.$$

Equivalentemente,

$$L\mathbf{1} = 0,$$

donde $\mathbf{1}$ es el vector de todos unos. Por tanto, 0 es siempre un autovalor de L, con autovector asociado $\mathbf{1}$.

2. *Para toda señal escalar $x \in \mathbb{R}^n$, se cumple*

$$x^T L x = \frac{1}{2} \sum_{i,j=1}^{n} A_{ij}(x_i - x_j)^2.$$

En particular, como $A_{ij} \geq 0$, se tiene $x^T L x \geq 0$ para todo $x \in \mathbb{R}^n$. Por tanto, L es semidefinida positiva y todos sus autovalores son no negativos. Esta expresión muestra que el Laplaciano mide variación local: si dos nodos conectados tienen valores muy distintos, el término $(x_i - x_j)^2$ contribuye mucho; si los valores son parecidos, contribuye poco.

3. *Si los autovalores de L se ordenan como*

$$0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n,$$

entonces λ_2 recibe el nombre de conectividad algebraica o spectral gap. *Este valor mide hasta qué punto el grafo está conectado: en particular, para grafos no dirigidos, $\lambda_2 > 0$ si y sólo si el grafo es conexo.*

4. *Si $d_{\text{máx}} = \text{máx}_i\, d_i$, entonces el mayor autovalor del Laplaciano satisface la cota*

$$\lambda_{\text{máx}} \leq 2d_{\text{máx}}.$$

Estas propiedades justifican el papel central del Laplaciano en el análisis espectral de grafos: sus autovalores son no negativos, el autovalor cero está asociado a las señales constantes y la forma cuadrática $x^T Lx$ mide la variación local de una señal sobre las aristas del grafo.

También se usan versiones normalizadas del Laplaciano. Cuando todos los grados son positivos, se define el Laplaciano normalizado simétrico como

$$L_{\text{sym}} = I - D^{-1/2} A D^{-1/2},$$

y el Laplaciano normalizado aleatorio como

$$L_{\text{rw}} = I - D^{-1} A.$$

El operador L_{sym} es simétrico para grafos no dirigidos con pesos simétricos, mientras que L_{rw} no lo es en general, salvo en casos particulares. Por tanto, cuando se requiera una base ortonormal de autovectores para definir la transformada de Fourier sobre grafos, se trabajará con el Laplaciano combinatorio $L = D - A$ o con L_{sym} [6, 14].

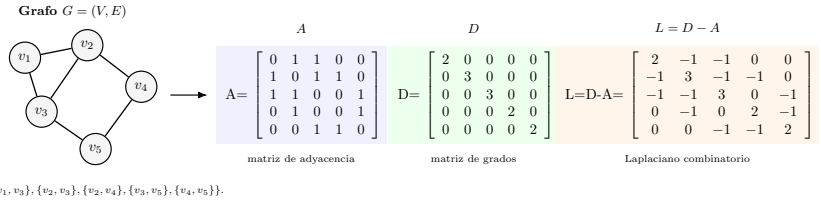

Figura 7: Representación matricial de un grafo no dirigido. La matriz de adyacencia A codifica las conexiones del grafo: una arista $\{v_i, v_j\}$ produce entradas $A_{ij} = A_{ji} = 1$. La matriz de grados D coloca el grado d_i de cada nodo en la diagonal. El Laplaciano combinatorio se obtiene como $L = D - A$, por lo que $L_{ii} = d_i$ y $L_{ij} = -1$ cuando v_i y v_j son adyacentes. Elaboración propia.

5.2. Señales sobre grafos

En una imagen, la entrada de una CNN puede interpretarse como una señal definida sobre una rejilla regular: a cada píxel se le asigna un valor, o varios valores si hay canales. En un grafo se adopta una idea análoga, pero el dominio ya no es una rejilla sino el conjunto de nodos V.

Una señal escalar sobre un grafo es una aplicación

$$x : V \longrightarrow \mathbb{R}.$$

Una vez fijado un orden de los nodos $V = \{v_1, \dots, v_n\}$, esta señal se representa como un vector

$$\mathbf{x} = \begin{bmatrix} x(v_1) \\ \vdots \\ x(v_n) \end{bmatrix} \in \mathbb{R}^n.$$

Más generalmente, si cada nodo tiene F características, una señal vectorial sobre el grafo es una aplicación

$$x : V \longrightarrow \mathbb{R}^F.$$

En forma matricial, las características de todos los nodos se apilan en una matriz

$$X = \begin{bmatrix} \mathbf{x}_{v_1}^T \\ \vdots \\ \mathbf{x}_{v_n}^T \end{bmatrix} \in \mathbb{R}^{n \times F},$$

donde la fila i-ésima contiene el vector de atributos del nodo v_i.

Esta notación permite interpretar una GNN como una transformación que recibe la estructura del grafo y sus señales:

$$(A, X) \longmapsto H,$$

donde $H \in \mathbb{R}^{n \times F'}$ contiene nuevas representaciones de los nodos. En una primera capa, normalmente se toma

$$H^{(0)} = X.$$

Después, cada capa actualiza las representaciones usando la estructura de vecindad. Intuitivamente, una capa GNN reemplaza la operación local de una CNN en una rejilla por una operación local en el vecindario del grafo.

El producto $A\mathbf{x}$ proporciona una primera forma algebraica de mezclar información local:

$$(A\mathbf{x})_i = \sum_{j=1}^{n} A_{ij} x_j = \sum_{v_j \in \mathcal{N}(v_i)} x_j.$$

Por tanto, $A\mathbf{x}$ asigna a cada nodo la suma de las señales de sus vecinos. De forma análoga, para señales vectoriales,

$$(AX)_i = \sum_{v_j \in \mathcal{N}(v_i)} \mathbf{x}_{v_j}.$$

Esta observación es importante porque anticipa la estructura básica de muchas GNN: agregar información de los vecinos y combinarla con la representación del nodo central.

El Laplaciano también tiene una interpretación local. Para una señal escalar $\mathbf{x}$, se tiene

$$(L\mathbf{x})_i = d_i x_i - \sum_{v_j \in \mathcal{N}(v_i)} x_j = \sum_{v_j \in \mathcal{N}(v_i)} (x_i - x_j).$$

Así, $L\mathbf{x}$ mide cuánto difiere el valor de un nodo respecto de los valores de sus vecinos. Esta idea es la base de muchas nociones de suavidad sobre grafos: una señal es suave si nodos conectados tienden a tener valores parecidos [14].

En el contexto de aprendizaje automático, además de atributos de nodos pueden existir atributos de aristas. Si cada arista (v_i, v_j) tiene un vector $\mathbf{e}_{ij} \in \mathbb{R}^{F_e}$, la entrada completa puede escribirse como

$$G = (V, E, X, E_f),$$

donde $E_f = \{\mathbf{e}_{ij} : (v_i, v_j) \in E\}$. Este caso es especialmente relevante cuando la relación entre dos nodos no solo indica conexión, sino también distancia, tiempo, coste, capacidad, tipo de enlace u otra información relacional.

5.3. Permutaciones, invariancia y equivarianza

Una diferencia esencial entre una imagen y un grafo es que los píxeles de una imagen tienen un orden geométrico natural, mientras que los nodos de un grafo no lo tienen en general. El orden $v_1, \ldots, v_n$ usado para construir A y X es una elección de etiquetado, no una propiedad intrínseca del grafo.

Por ejemplo, si se reordenan los nodos, cambia la forma matricial de la adyacencia, pero no cambia el grafo representado. Sea $P \in \{0,1\}^{n \times n}$ una matriz de permutación. Si se cambia el etiquetado de los nodos mediante P, entonces la matriz de atributos pasa a ser

$$X' = PX,$$

y la matriz de adyacencia pasa a ser

$$A' = PAP^T.$$

Las matrices A y A' representan el mismo grafo con distinto orden de nodos. Esta idea está estrechamente relacionada con la noción clásica de isomorfismo de grafos: dos grafos simples son isomorfos si existe una biyección entre sus conjuntos de vértices que preserva la adyacencia [13].

En aprendizaje sobre grafos, esta falta de orden canónico impone una condición estructural sobre los modelos. Si una arquitectura produce una representación por nodo, debe ser equivariante a permutaciones. Es decir, si cambiamos el orden de los nodos en la entrada, la salida debe cambiar de la misma forma:

$$\Phi(PAP^T, PX) = P\Phi(A, X).$$

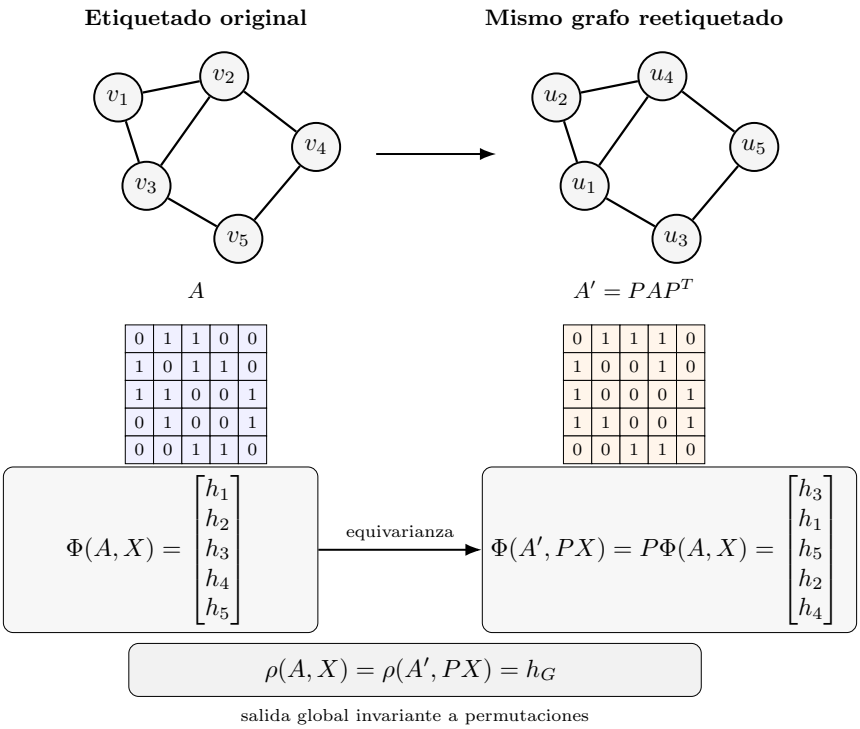

Figura 8: Efecto de una permutación del etiquetado de los nodos. Los dos dibujos representan el mismo grafo, aunque sus matrices de adyacencia sean distintas. Una función con salida por nodo debe ser equivariante: si la entrada se reordena, la salida se reordena del mismo modo. En cambio, una función con salida global debe ser invariante: la representación del grafo completo no debe depender del etiquetado elegido. Elaboración propia.

Aquí $\Phi(A, X) \in \mathbb{R}^{n \times F'}$ representa una función que devuelve una salida por nodo. La ecuación anterior dice que el modelo no debe depender del nombre o posición arbitraria del nodo en la matriz, sino únicamente de la estructura relacional.

En cambio, si la tarea produce una única salida para todo el grafo, como en clasificación de grafos, la función debe ser invariante a permutaciones. Es decir,

$$\rho(PAP^T, PX) = \rho(A, X).$$

En este caso, reordenar los nodos no debe modificar la predicción global del grafo.

Esta distinción será fundamental en las GNN. Las capas de message passing suelen ser equivariantes a permutaciones porque actualizan cada nodo usando operaciones locales compartidas. Después, para obtener una representación global del grafo, se aplica una operación de lectura o *readout* invariante, por ejemplo suma, media o máximo:

$$\mathbf{h}_G = \mathrm{READOUT}\left(\{\mathbf{h}_v : v \in V\}\right).$$

El uso de agregadores invariantes al orden no es un detalle técnico, sino una necesidad matemática: el vecindario $\mathcal{N}(v)$ no viene ordenado. Por tanto, una operación sobre vecinos debe producir el mismo resultado independientemente del orden en que se enumeren. Esta es la diferencia central con la convolución en rejillas, donde sí existe una noción fija de desplazamiento espacial.

5.4. Tareas de aprendizaje sobre grafos

Las tareas de aprendizaje sobre grafos se distinguen por el nivel al que se realiza la predicción. Esta distinción es relevante porque determina qué tipo de salida debe producir el modelo y qué simetría debe respetar: equivarianza a permutaciones cuando la salida está asociada a nodos o aristas, e invariancia cuando la salida representa al grafo completo [16, 17].

Predicción a nivel de nodo. En este caso se dispone de un grafo $G = (V, E)$, atributos de nodo $X \in \mathbb{R}^{|V| \times F}$ y, para algunos o todos los nodos, etiquetas y_v. El objetivo es aprender una función

$$f_\theta(G, X)_v \in \mathcal{Y}, \qquad v \in V,$$

que asigne una salida a cada nodo. La arquitectura debe ser equivariante a permutaciones: si se reordena el etiquetado de los nodos, las predicciones deben reordenarse del mismo modo.

Predicción a nivel de arista o enlace. Aquí la salida está asociada a pares de nodos. En predicción de enlaces se estima, por ejemplo, la probabilidad de existencia de una arista:

$$s_\theta(u, v) = \Pr\big((u, v) \in E \mid G, X\big).$$

En otros problemas la arista existe, pero se busca clasificarla o regresionar una propiedad asociada a ella, como coste, tipo, capacidad o tiempo.

Predicción a nivel de grafo. En clasificación o regresión de grafos, cada muestra es un grafo completo $G_i = (V_i, E_i, X_i)$, posiblemente con atributos de arista, y se busca una salida global

$$f_\theta(G_i) \in \mathcal{Y}.$$

En este caso, el modelo debe combinar capas equivariantes a nivel de nodo con un operador de lectura global invariante a permutaciones, por ejemplo suma, media, máximo o atención global. Esta es la formulación natural cuando cada instancia experimental se representa como un grafo, una ruta o un subgrafo.

Relación con la experimentación. La elección de la tarea condiciona los modelos y las métricas. En clasificación de nodos son habituales particiones por nodos dentro de uno o varios grafos; en clasificación de grafos se trabaja con particiones por grafos completos. Benchmarks como OGB proporcionan conjuntos de datos, particiones y evaluadores para varias tareas de aprendizaje sobre grafos, mientras que TUDataset reúne numerosos conjuntos para clasificación y regresión de grafos [28, 29].

6. De CNN a GNN

En las secciones anteriores se han introducido los dos ingredientes necesarios para formular la transición desde CNN a GNN. Por un lado, una CNN explota la estructura regular de una rejilla mediante conectividad local, compartición de pesos y equivarianza a traslaciones [5, 3]. Por otro lado, un grafo proporciona una estructura relacional irregular, descrita por nodos, aristas, matrices asociadas y señales definidas sobre sus vértices [13, 14]. La pregunta central de esta sección es qué significa extender la idea de convolución cuando el dominio deja de ser una rejilla regular y pasa a ser un grafo arbitrario [15, 43].

Históricamente, esta transición se ha abordado principalmente desde dos puntos de vista. El primero es el enfoque espectral, que define filtros sobre grafos mediante el Laplaciano y la transformada de Fourier sobre grafos [6, 14, 46]. El segundo es el enfoque espacial, que interpreta la convolución como una operación local de agregación de información en vecindarios y que conduce al paradigma de *message passing* [16, 17, 44]. En esta sección se presentan ambos enfoques como pasos conceptuales hacia las arquitecturas GNN que se estudiarán posteriormente.

6.1. La imagen como caso particular de dominio regular

Una imagen puede interpretarse como una señal multicanal definida sobre una rejilla bidimensional. Cada posición espacial (u, v) posee un vector de carac-

terísticas formado por sus canales, y sus vecinos se identifican mediante desplazamientos fijos $(\Delta u, \Delta v)$. Esta estructura regular permite definir filtros locales cuyos coeficientes se comparten en todas las posiciones de la imagen [5, 3].

Sea $\mathcal{K} \subset \mathbb{Z}^2$ el conjunto finito de desplazamientos del campo receptivo, por ejemplo $\mathcal{K} = \{-r, \dots, r\} \times \{-r, \dots, r\}$ para un filtro cuadrado de tamaño $(2r + 1) \times (2r + 1)$. Entonces, omitiendo canales y sesgos para aligerar la notación, una convolución discreta sobre una rejilla puede escribirse como [5]

$$(K * X)_{u,v} = \sum_{\Delta u, \Delta v} K_{\Delta u, \Delta v} X_{u+\Delta u, v+\Delta v}.$$

Esta expresión muestra tres propiedades fundamentales de las CNN: localidad, porque la salida en (u, v) depende solo de un entorno de esa posición; compartición de pesos, porque el mismo filtro K se aplica en todas las posiciones; y estructura geométrica, porque cada coeficiente $K_{\Delta u, \Delta v}$ está asociado a un desplazamiento espacial concreto [5, 15].

Desde esta perspectiva, una imagen puede verse como un caso particular de grafo: los píxeles son nodos y las aristas conectan píxeles vecinos. Formalmente, una rejilla $H \times W$ puede representarse como un grafo $G_{\text{grid}} = (V, E)$, con

$$V = \{1, \dots, H\} \times \{1, \dots, W\},$$

y con aristas entre posiciones separadas por desplazamientos de un conjunto $\mathcal{K}$, por ejemplo los cuatro vecinos horizontales y verticales. Sin embargo, se trata de un grafo altamente regular, ya que los nodos interiores tienen el mismo patrón de vecindad y los vecinos están ordenados por desplazamientos espaciales [15, 43]. Esta regularidad es precisamente lo que permite definir un kernel $k_H \times k_W$ con posiciones relativas fijas.

6.2. La dificultad de extender un kernel convolucional a grafos

En un grafo general, un nodo v tiene asociado un vecindario

$$\mathcal{N}(v) = \{u \in V : \{u, v\} \in E\}.$$

La analogía con las CNN sugiere que la representación de v debería actualizarse a partir de la información contenida en sus vecinos. Sin embargo, a diferencia de una rejilla, en un grafo arbitrario el número de vecinos puede variar de un nodo a otro y no existe un orden canónico entre ellos [13, 15, 43].

Esta ausencia de orden es una diferencia estructural esencial. En una imagen tiene sentido hablar del vecino superior, inferior o diagonal de un píxel, pero en un grafo general no existe una noción global de dirección que permita asignar un coeficiente distinto del kernel a "la primera", "la segunda" o "la tercera" posición del vecindario. Por tanto, el kernel convolucional de una CNN no puede trasladarse literalmente a un grafo arbitrario [15, 43].

La consecuencia matemática es que la suma ordenada de la convolución clásica debe sustituirse por una operación definida sobre conjuntos o multiconjuntos de vecinos:

$$\mathrm{AGG}\{h_u : u \in \mathcal{N}(v)\}.$$

Para que la operación no dependa del etiquetado arbitrario de los nodos, el agregador debe ser invariante al orden de enumeración de los vecinos. Esta condición es central en las GNN modernas y aparece explícitamente en el marco de *message passing* [16, 21, 22].

Así, la transición de CNN a GNN no consiste en copiar un filtro $k \times k$, sino en conservar los principios estructurales de la convolución: localidad, compartición de parámetros y compatibilidad con las simetrías del dominio. En imágenes, la simetría relevante es la traslación; en grafos, la simetría relevante es la permutación del etiquetado de los nodos [15, 49].

6.3. Enfoque espectral: filtros definidos mediante el Laplaciano

Una primera forma de definir convoluciones sobre grafos parte de la teoría espectral. En procesamiento clásico de señales, la convolución puede interpretarse como una operación de filtrado en el dominio de Fourier. En grafos no dirigidos, el papel del operador frecuencial lo desempeña el Laplaciano del grafo, cuya estructura espectral permite definir una transformada de Fourier sobre grafos [6, 14].

Para definir dicha transformada usaremos un Laplaciano simétrico, por ejemplo el Laplaciano combinatorio

$$L = D - A$$

o el Laplaciano normalizado simétrico

$$L_{\mathrm{sym}} = I - D^{-1/2} A D^{-1/2}.$$

En ambos casos, por la Proposición 5.1, el Laplaciano combinatorio de un grafo no dirigido es simétrico y semidefinido positivo, por lo que admite una base ortonormal de autovectores y autovalores reales no negativos.

$$L = U \Lambda U^{\top}, \tag{17}$$

donde las columnas de U son autovectores de L y Λ es la matriz diagonal de autovalores.

Para una señal $x \in \mathbb{R}^n$ definida sobre los nodos, la transformada de Fourier sobre el grafo se define como

$$\hat{x} = U^{\top} x,$$

y la transformada inversa como

$$x = U \hat{x}.$$

Esta construcción interpreta los autovectores del Laplaciano como modos frecuenciales adaptados a la estructura del grafo.[6, 14].

En este marco, un filtro espectral parametrizado puede definirse como una función g_θ,donde $g_\theta(\Lambda)$ denota la matriz diagonal

$$g_\theta(\Lambda) = \mathrm{diag}(g_\theta(\lambda_1), \ldots, g_\theta(\lambda_n)).$$

aplicada a los autovalores del Laplaciano:

$$g_\theta(L)x = U g_\theta(\Lambda) U^T x.$$

La expresión anterior generaliza la idea de filtrado frecuencial: el filtro modifica las componentes de la señal en la base espectral inducida por el grafo [14, 46].

El enfoque espectral es matemáticamente natural, pero presenta dificultades prácticas. En primer lugar, la diagonalización del Laplaciano puede ser costosa para grafos grandes. En segundo lugar, un filtro definido mediante los autovectores puede depender fuertemente del grafo concreto, dificultando su transferencia a grafos con estructuras distintas [46, 9]. Para resolver parte de estas limitaciones, Defferrard, Bresson y Vandergheynst propusieron filtros localizados mediante aproximaciones polinomiales de Chebyshev, evitando calcular explícitamente la descomposición espectral completa y obteniendo operaciones locales sobre el grafo [9].

La Graph Convolutional Network de Kipf y Welling puede interpretarse como una aproximación localizada de primer orden, derivada a partir de una simplificación de los filtros espectrales de Chebyshev. Su formulación conduce a una regla de propagación basada en la matriz de adyacencia con autolazos y una normalización por grados, lo que proporciona una capa eficiente y ampliamente utilizada para aprendizaje semi-supervisado sobre grafos [18, 17].

6.4. Enfoque espacial: agregación local y message passing

El enfoque espacial adopta una interpretación más directa de la convolución: una operación local que actualiza la representación de cada nodo combinando información de su vecindario. En lugar de definir filtros en la base espectral del Laplaciano, este enfoque especifica explícitamente cómo se envían, agregan y transforman mensajes entre nodos conectados [16, 17, 44].

Sea $h_v^{(k)} \in \mathbb{R}^{d_k}$ la representación del nodo v en la capa k, con inicialización

$$h_v^{(0)} = x_v.$$

Una capa de *message passing* puede describirse mediante tres operaciones. Primero, para cada arista (u, v), se calcula un mensaje

$$m_{u\to v}^{(k)} = M_\theta^{(k)} \left(h_u^{(k)}, h_v^{(k)}, e_{uv} \right),$$

donde $M_\theta^{(k)}$ es una función parametrizada y e_{uv} representa, si existe, el atributo de la arista (este argumento puede omitirse o tomarse como una constante si el

grafo no tiene atributos de arista)[16]. En grafos no dirigidos, puede entenderse que cada arista $\{u, v\}$ induce dos mensajes dirigidos, $u \to v$ y $v \to u$.

Después, los mensajes recibidos por v se combinan mediante un operador de agregación

$$m_v^{(k)} = \bigoplus_{u \in \mathcal{N}(v)} m_{u \to v}^{(k)}.$$

El operador $\bigoplus$ debe ser invariante a permutaciones del vecindario, ya que el orden en que se enumeran los vecinos no forma parte de la estructura del grafo. Ejemplos habituales son la suma, la media y el máximo; también pueden emplearse agregaciones ponderadas, como ocurre en mecanismos de atención sobre grafos [16, 22, 33].

Finalmente, el nodo actualiza su representación mediante una función parametrizada:

$$h_v^{(k+1)} = U_\theta^{(k)} \left(h_v^{(k)}, m_v^{(k)} \right).$$

Esta plantilla permite ver muchas arquitecturas GNN como instancias de un mismo principio: elección de una función de mensaje, un agregador invariante y una función de actualización [16, 17, 49].

La localidad se conserva porque, en una capa, cada nodo recibe información solo de sus vecinos inmediatos. Al apilar varias capas, el campo receptivo crece: tras una capa, la representación de v puede depender de nodos a distancia uno; tras dos capas, de nodos a distancia dos; y, en general, tras K capas, de nodos en un entorno de K-saltos [16, 44, 17].

La compartición de parámetros también se conserva: las mismas funciones $M_\theta^{(k)}$ y $U_\theta^{(k)}$ se aplican a todos los nodos y aristas de una misma capa. Esta es la analogía con el uso de un mismo kernel en todas las posiciones de una imagen, aunque en grafos la operación debe respetar la simetría por permutación y no la simetría por traslación [5, 15, 43].

7. Arquitectura GNN

7.1. Graph Convolutional Network (GCN)

La Graph Convolutional Network (GCN) de Kipf y Welling es una de las arquitecturas de referencia en aprendizaje profundo sobre grafos. Puede entenderse como una aproximación localizada de primer orden a filtros convolucionales espectrales, pero su forma final admite una interpretación espacial sencilla: cada nodo actualiza su representación combinando la información de sus vecinos y de sí mismo mediante una normalización por grados [18, 17].

Sea $A \in \mathbb{R}^{n \times n}$ la matriz de adyacencia de un grafo no dirigido y sea $H^{(k)} \in \mathbb{R}^{n \times d_k}$ la matriz de representaciones de nodos en la capa k. Se añaden autolazos mediante

$$\tilde{A} = A + I_n,$$

y se define la matriz de grados asociada

$$\tilde{D}_{ii} = \sum_{j=1}^{n} \tilde{A}_{ij}.$$

La regla de propagación de una capa GCN es

$$H^{(k+1)} = \sigma\Big(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(k)}W^{(k)}\Big), \tag{18}$$

donde $W^{(k)} \in \mathbb{R}^{d_k \times d_{k+1}}$ es una matriz de parámetros compartida y σ es una función de activación.

La adición de autolazos permite que cada nodo conserve parte de su representación previa. La normalización simétrica evita que los nodos de grado alto dominen la agregación y puede interpretarse como un suavizado laplaciano de las representaciones sobre el grafo. Esta propiedad explica parte de la eficacia de GCN, pero también anticipa una de sus limitaciones: al apilar muchas capas, las representaciones pueden tender a hacerse demasiado parecidas, fenómeno conocido como *over-smoothing* [19, 20].

En forma nodo a nodo, la ecuación (18) puede escribirse como

$$h_i^{(k+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i)\cup\{i\}} \frac{1}{\sqrt{\tilde{d}_i \tilde{d}_j}}\, h_j^{(k)} W^{(k)}\right). \tag{19}$$

Por tanto, GCN es un caso particular de paso de mensajes donde los pesos de agregación están fijados por la estructura del grafo, no aprendidos por un mecanismo de atención.

El coste de una capa implementada con matrices dispersas escala linealmente con el número de aristas, más el coste de la transformación densa de atributos. De forma aproximada, para dimensión oculta d, el coste es

$$O(|E|d + nd^2),$$

según el orden de las operaciones y las dimensiones de entrada y salida. Esta simplicidad convierte a GCN en un baseline neuronal obligatorio frente a modelos más expresivos como GIN, GAT o GGNN. La documentación de PyTorch Geometric implementa esta regla mediante ‘GCNConv‘, incluyendo autolazos y normalización simétrica por defecto [48].

7.2. GraphSAGE

GraphSAGE se diseñó como un marco inductivo para aprender representaciones de nodos mediante funciones agregadoras de vecindario, en lugar de aprender un embedding independiente para cada nodo del grafo de entrenamiento [47]. Esta diferencia es importante cuando, evaluando datos ajenos al entrenamiento, aparecen nodos, grafos o subgrafos no observados.

Sea $h_v^{(0)} = x_v$ la representación inicial del nodo v. En la capa k, GraphSAGE calcula primero una representación agregada del vecindario:

$$h_{\mathcal{N}(v)}^{(k)} = \mathrm{AGG}^{(k)}\left(\left\{h_u^{(k-1)} : u \in \mathcal{N}(v)\right\}\right), \tag{20}$$

donde $\mathrm{AGG}^{(k)}$ debe ser invariante al orden de los vecinos. Después concatena dicha representación con la del mismo nodo de la capa anterior:

$$h_v^{(k)} = \sigma\Big(W^{(k)}\left[h_v^{(k-1)} \,\|\, h_{\mathcal{N}(v)}^{(k)}\right]\Big). \tag{21}$$

Opcionalmente se normaliza $h_v^{(k)}$ a norma unitaria.

El artículo original considera agregadores como media, LSTM y *pooling*. El agregador de media produce una arquitectura sencilla y eficiente:

$$h_{\mathcal{N}(v)}^{(k)} = \frac{1}{|\mathcal{N}(v)|}\sum_{u\in\mathcal{N}(v)} h_u^{(k-1)}.$$

El agregador por *pooling* aplica primero una transformación no lineal a cada vecino y después una operación simétrica como el máximo:

$$h_{\mathcal{N}(v)}^{(k)} = \max_{u\in\mathcal{N}(v)} \sigma\Big(W_{\mathrm{pool}} h_u^{(k-1)} + b\Big),$$

donde el máximo se toma coordenada a coordenada.

En grafos grandes, GraphSAGE suele muestrear un número fijo de vecinos por capa, lo que controla el coste computacional y permite entrenamiento por mini-batches. Esta propiedad lo hace especialmente útil como baseline inductivo. En PyTorch Geometric, 'SAGEConv' implementa el operador GraphSAGE con agregación configurable [23].

7.3. Graph Isomorphism Network (GIN)

7.3.1. Contexto: agregación de vecindarios, multiconjuntos y poder expresivo

Sea $G = (V, E)$ un grafo (dirigido o no dirigido) con $|V| = n$ nodos. Supondremos atributos en nodos $\mathbf{x}_v \in \mathbb{R}^F$ para cada $v \in V$ y, cuando proceda, atributos en aristas $\mathbf{e}_{u,v} \in \mathbb{R}^D$ para cada $(u, v) \in E$. Un *modelo de message passing* actualiza representaciones ocultas de nodos $\mathbf{h}_v^{(k)} \in \mathbb{R}^{d_k}$ por capas $k = 0, 1, \ldots, K$, inicializando $\mathbf{h}_v^{(0)} = \mathbf{x}_v$, mediante una regla de la forma

$$\mathbf{h}_i^{(k)} \;=\; \gamma^{(k)}\left(\mathbf{h}_i^{(k-1)},\; \bigoplus_{j\in\mathcal{N}(i)} \phi^{(k)}\Big(\mathbf{h}_i^{(k-1)}, \mathbf{h}_j^{(k-1)}, \mathbf{e}_{j,i}\Big)\right), \tag{22}$$

donde $\mathcal{N}(i)$ denota el vecindario de i, $\bigoplus$ es un agregador *invariante a permutaciones* (p.ej. suma, media, máximo), y $\phi^{(k)}, \gamma^{(k)}$ suelen ser MLPs.

Aunque $N(v)$ es un conjunto de nodos en un grafo simple, la colección de representaciones $\{h_u : u \in N(v)\}$ debe tratarse como un multiconjunto, ya que distintos vecinos pueden compartir la misma etiqueta o embedding. Por ello, el agregador de una GNN no debe depender del orden en el que se enumeran los vecinos, pero sí debe poder preservar multiplicidades cuando la tarea lo requiera [21, 22].

Definición 7.1 (Multiconjunto)**.** *Un* multiconjunto X *es un par* (S, m) *donde* S *es el conjunto de elementos distintos y* $m : S \to \mathbb{N}_{\geq 1}$ *asigna la multiplicidad de cada elemento. Equivalentemente, puede verse como una colección* $\{x_1, \ldots, x_t\}$ *donde elementos idénticos pueden repetirse.*

Dado que el orden de los vecinos es arbitrario, cualquier agregación debe ser invariante a permutaciones, es decir, una función sobre multiconjuntos. En tareas donde la estructura importa (p.ej. clasificación de grafos/rutas), queremos que la agregación sea lo más *discriminativa* posible: idealmente, multiconjuntos distintos deberían mapear a representaciones distintas (inyectividad).

7.3.2. Relación con Weisfeiler–Lehman (1-WL) y un criterio de optimalidad

El test de isomorfismo de Weisfeiler–Lehman en su versión 1-dimensional (1-WL, también llamado *color refinement*) itera etiquetas discretas $l_v^{(k)}$ actualizando

$$l_v^{(k)} \;=\; \mathrm{HASH}\Big(l_v^{(k-1)},\; \{\!\{\, l_u^{(k-1)} : u \in \mathcal{N}(v) \,\}\!\}\Big), \tag{23}$$

donde $\{\!\{\cdot\}\!\}$ denota multiconjunto y HASH se toma inyectiva (multiconjuntos distintos $\mapsto$ etiquetas distintas). Intuitivamente, $l_v^{(k)}$ codifica el *subárbol enraizado* de altura k en v.

El marco teórico de [21] muestra dos hechos fundamentales:

Teorema 7.1 (Cota superior WL para GNNs de agregación)**.** *Sea* $\mathcal{A}$ *una GNN estándar de paso de mensajes cuyas representaciones pertenecen a espacios euclidianos de dimensión finita, que actualiza representaciones mediante funciones continuas, o al menos medibles, de agregación invariante a permutaciones de vecindarios, y que no utiliza identificadores de nodos, codificaciones posicionales ni mecanismos de orden superior. Si* $\mathcal{A}$ *asigna representaciones de grafo distintas a dos grafos, entonces el test* 1*-WL también distingue dichos grafos.*

Teorema 7.2 (Suficiencia de inyectividad para igualar a 1-WL)**.** *Considérese una GNN con actualización*

$$\mathbf{h}_v^{(k)} = \varphi\Big(\mathbf{h}_v^{(k-1)},\; f\big(\{\!\{\mathbf{h}_u^{(k-1)} : u \in \mathcal{N}(v)\}\!\}\big)\Big),$$

donde f *(sobre multiconjuntos) y* φ *son inyectivas. Si además el* readout *a nivel de grafo sobre el multiconjunto* $\{\!\{\mathbf{h}_v^{(k)} : v \in V\}\!\}$ *es inyectivo, entonces, con suficientes capas, la GNN distingue todos los pares de grafos que distingue 1-WL.*

Los Teoremas 7.1–7.2 justifican el objetivo arquitectónico: *diseñar agregación y readout inyectivos*. La Graph Isomorphism Network (GIN) es una realización simple y teóricamente motivada de este principio.

7.3.3. Suma como agregador universal de multiconjuntos

La elección del agregador $\bigoplus$ no es casual. La media o el máximo pueden colapsar multiconjuntos distintos (pérdida de multiplicidades o de escala). En cambio, bajo hipótesis suaves (atributos en un universo contable y multiconjuntos de tamaño acotado), la suma permite representar funciones inyectivas sobre multiconjuntos.

[Inyectividad de la suma tras un mapeo adecuado] Supóngase que el espacio de entrada $\mathcal{X}$ es contable y que consideramos multiconjuntos $X \subset \mathcal{X}$ con cardinal acotado. Entonces existe $f : \mathcal{X} \to \mathbb{R}^m$ tal que

$$\Psi(X) \;=\; \sum_{x \in X} f(x)$$

es inyectiva en el conjunto de esos multiconjuntos. Además, toda función g sobre multiconjuntos puede escribirse como

$$g(X) = \phi\left(\sum_{x \in X} f(x)\right)$$

para cierta ϕ.

Este lema es la versión "multiconjunto" de la idea de *Deep Sets* (funciones invariantes a permutaciones), y su consecuencia práctica es clara: si ϕ y f pueden aproximarse con MLPs, entonces podemos aproximar funciones muy generales (incluyendo inyectivas) sobre vecindarios.

Un detalle crucial en grafos es que la actualización no depende sólo del multiconjunto vecino, sino también del nodo central. El siguiente corolario muestra una forma simple de *separar* el nodo central del vecindario sin perder inyectividad.

Corolario 7.1 (Separación inyectiva del nodo central)**.** *Bajo las hipótesis del Lema 7.3.3, existe f tal que, para infinitos valores de ε, la aplicación*

$$(c, X) \;\longmapsto\; (1+\varepsilon)\, f(c) + \sum_{x \in X} f(x)$$

es inyectiva sobre pares (c, X), donde $c \in \mathcal{X}$ es el elemento central y X el multiconjunto vecino.

7.3.4. Definición de la capa GIN

La GIN toma exactamente la forma sugerida por el Corolario 7.1, instanciando f y ϕ (composición) mediante una MLP. En la capa k:

$$\mathbf{h}_v^{(k)} \;=\; \mathrm{MLP}^{(k)}\left((1+\varepsilon^{(k)})\,\mathbf{h}_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(k-1)}\right), \tag{24}$$

donde $\varepsilon^{(k)} \in \mathbb{R}$ puede fijarse (p.ej. $\varepsilon^{(k)} = 0$) o aprenderse. El término $(1 + \varepsilon^{(k)})\mathbf{h}_v^{(k-1)}$ actúa como un *peso de auto-bucle* (self-loop) parametrizable, permitiendo distinguir el aporte del nodo central del de sus vecinos.

En notación matricial, si $\mathbf{X} \in \mathbb{R}^{n \times d}$ apila las representaciones por filas y $\mathbf{A}$ es la matriz de adyacencia (sin auto-bucles), entonces

$$\mathbf{X}' \;=\; h_\Theta\big((\mathbf{A} + (1+\varepsilon)\mathbf{I})\mathbf{X}\big)\,, \tag{25}$$

donde h_Θ es la MLP compartida por capa.

Observación 7.1 (Por qué MLP y por qué suma). La suma preserva multiplicidades del multiconjunto, condición necesaria para alcanzar el poder de 1-WL en general. La MLP (frente a una sola capa lineal) es esencial para aproximar funciones suficientemente ricas sobre multiconjuntos, incluyendo funciones inyectivas.

7.3.5. Readout a nivel de grafo y clasificación

Para tareas de clasificación de grafos, o para problemas en los que cada instancia se representa como un grafo, una ruta o un subgrafo, se requiere un operador de lectura global que produzca un embedding h_G a partir del multiconjunto de representaciones finales de los nodos. Dicho operador debe ser invariante a permutaciones. Una opción teóricamente coherente con el Teorema 7.2 es la suma:

$$\mathbf{s}^{(k)} \;=\; \sum_{v \in V} \mathbf{h}_v^{(k)}.$$

Además, conviene usar información de *todas* las profundidades, ya que distintas capas capturan subestructuras de distinto radio. Así, una arquitectura habitual en GIN concatena readouts por capa:

$$\mathbf{h}_G \;=\; \mathrm{CONCAT}\Big(\mathbf{s}^{(0)}, \mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(K)}\Big). \tag{26}$$

Finalmente, un clasificador (p.ej. una MLP final) produce $\hat{\mathbf{y}}_G = \mathrm{softmax}(\mathrm{MLP}_{\mathrm{out}}(\mathbf{h}_G))$ y se ajusta con pérdida de entropía cruzada.

7.3.6. Extensión natural cuando hay atributos en aristas: GINE

La GIN original no utiliza atributos de arista $e_{j,i}$. Cuando las aristas poseen información propia, como distancia, tiempo, coste o tipo de relación, puede usarse la variante GINE (GIN with Edge features):

$$h_i' = h_\Theta\left((1+\epsilon)h_i + \sum_{j \in \mathcal{N}(i)} \mathrm{ReLU}(h_j + e_{j,i})\right). \tag{27}$$

Si la dimensionalidad de los atributos de arista no coincide con la de los nodos, se aplica una transformación lineal previa a $e_{j,i}$. Esta es la convención implementada en `GINEConv` de PyTorch Geometric mediante el parámetro `edge_dim` [27, 32].

7.3.7. Correspondencia explícita con la interfaz `MessagePassing` en PyTorch Geometric

En PyTorch Geometric, una capa de message passing se especifica mediante las funciones `message` (implementa ϕ), `aggregate` (implementa $\bigoplus$) y `update` (implementa γ), siguiendo (22). La `GINConv` concreta esta plantilla tomando agregación `."add"` y un mensaje esencialmente identidad $\phi(\cdot) = \mathbf{h}_j$, mientras que la actualización corresponde a (24)–(25). Para incorporar atributos en aristas de manera directa, `GINEConv` implementa (27).

7.3.8. Motivación y planteamiento

Sea un grafo (posiblemente dirigido) $G = (V, E)$, con $n = |V|$. A cada nodo $i \in V$ se asocia un vector de atributos inicial $\mathbf{x}_i \in \mathbb{R}^F$. (Opcionalmente, a cada arista $(j, i) \in E$ se le asocia un vector de atributos $\mathbf{e}_{ji} \in \mathbb{R}^{F_e}$.)

Una familia amplia de arquitecturas GNN define, para cada capa $\ell = 0, 1, \ldots, L-1$, representaciones ocultas $\mathbf{h}_i^{(\ell)} \in \mathbb{R}^{d_\ell}$ con $\mathbf{h}_i^{(0)} := \mathbf{x}_i$, y una regla de actualización local del tipo

$$\mathbf{h}_i^{(\ell+1)} = \mathrm{UPDATE}^{(\ell)}\Big(\mathbf{h}_i^{(\ell)}, \mathrm{AGG}^{(\ell)}\big(\{\mathbf{h}_j^{(\ell)} : j \in \mathcal{N}(i)\}\big)\Big),$$

donde $\mathcal{N}(i) = \{j \in V : (j, i) \in E\}$ denota el vecindario entrante (en no dirigido, $(j, i) \in E \Leftrightarrow (i, j) \in E$). En muchos modelos convolucionales sobre grafos, AGG es un promedio normalizado o una suma con pesos fijados por la estructura. La idea central de GAT consiste en *aprender* los pesos de agregación con un mecanismo de atención dependiente de las características.

7.3.9. Capa de atención sobre grafos (GAT)

Transformación lineal compartida. Fijemos una dimensión $d' = d_{\ell+1}$. La capa parte de $\mathbf{h}_i \in \mathbb{R}^d$ (omitimos el superíndice ℓ para aligerar) y aplica una transformación lineal compartida

$$\mathbf{z}_i = \mathbf{W}\mathbf{h}_i \in \mathbb{R}^{d'}, \qquad \mathbf{W} \in \mathbb{R}^{d' \times d}.$$

Puntuación atencional (masked self-attention). Para cada arista $(j, i) \in E$ (y, típicamente, también el autolazo (i, i)), se define una *puntuación* escalar e_{ij} que mide la relevancia de j para i. En la formulación estándar de GAT [33]:

$$e_{ij} = \mathrm{LeakyReLU}\big(\mathbf{a}^\top [\mathbf{z}_i \,\|\, \mathbf{z}_j]\big), \qquad \mathbf{a} \in \mathbb{R}^{2d'}, \tag{28}$$

donde $[\cdot\|\cdot]$ denota concatenación. En la práctica se usa LeakyReLU con pendiente negativa típica 0,2.

Normalización por softmax en el vecindario. Las puntuaciones se normalizan sobre el vecindario (atención enmascarada):

$$\alpha_{ij} = \mathrm{softmax}_{j\in\mathcal{N}(i)\cup\{i\}}(e_{ij}) = \frac{\exp(e_{ij})}{\sum\limits_{k\in\mathcal{N}(i)\cup\{i\}}\exp(e_{ik})}. \tag{29}$$

Así, para cada i, $\alpha_{ij} \geq 0$ y $\sum_{j\in\mathcal{N}(i)\cup\{i\}} \alpha_{ij} = 1$, por lo que $\alpha_{i\cdot}$ define una distribución sobre los vecinos de i.

Agregación y no linealidad. La salida de la capa (una cabeza de atención) se define como

$$\mathbf{h}'_i = \sigma\left(\sum_{j\in\mathcal{N}(i)\cup\{i\}} \alpha_{ij}\,\mathbf{z}_j\right) = \sigma\left(\sum_{j\in\mathcal{N}(i)\cup\{i\}} \alpha_{ij}\,\mathbf{W}\mathbf{h}_j\right), \tag{30}$$

donde σ es una activación (p.ej. ELU en capas internas y softmax/sigmoide al final).

7.3.10. Interpretación como Message Passing

La ecuación (30) encaja en el marco MPNN (message passing): definiendo mensajes

$$\mathbf{m}_{ij} := \alpha_{ij}\,\mathbf{W}\mathbf{h}_j \in \mathbb{R}^{d'},$$

la agregación es suma $\mathbf{m}_i := \sum_{j\in\mathcal{N}(i)\cup\{i\}} \mathbf{m}_{ij}$ y la actualización es $\mathbf{h}'_i = \sigma(\mathbf{m}_i)$. En GAT la *clave* es que los pesos α_{ij} se obtienen mediante un mecanismo atencional dependiente de $\mathbf{h}_i$ y $\mathbf{h}_j$, en lugar de estar fijados por normalizaciones de grado como en GCN.

7.3.11. Invarianza a permutaciones en el vecindario

Un requisito estructural habitual es que la capa sea invariante a cualquier reordenación de los vecinos. Esta propiedad se satisface:

Proposición 7.1 (Invarianza por permutación local)**.** *Sea π una permutación de $\mathcal{N}(i)\cup\{i\}$. Si se reindexa el vecindario en* (28)–(30) *sustituyendo j por $\pi(j)$, entonces $\mathbf{h}'_i$ no cambia.*

Demostración. La colección de puntuaciones $\{e_{ij}\}_{j\in\mathcal{N}(i)\cup\{i\}}$ se reordena al aplicar la permutación π. Sin embargo, el softmax se calcula sobre todo el vecindario:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k\in\mathcal{N}(i)\cup\{i\}}\exp(e_{ik})}.$$

Como el denominador es una suma sobre el conjunto completo de vecinos, no depende del orden en que estos se enumeren. Por tanto, si las puntuaciones e_{ij} se reordenan mediante π, los coeficientes α_{ij} se reordenan de la misma forma.

En este sentido, el softmax es equivariante a permutaciones del conjunto sobre el que normaliza.

Finalmente, la actualización

$$\mathbf{h}_i' = \sigma\left(\sum_{j\in\mathcal{N}(i)\cup\{i\}} \alpha_{ij} W\mathbf{h}_j\right)$$

no cambia, ya que la suma es conmutativa. Por tanto, el resultado no depende del orden elegido para enumerar los vecinos. □

7.3.12. Atención multi-cabeza

Para estabilizar y enriquecer la representación, GAT emplea K cabezas de atención independientes [33]. Para cada cabeza $k = 1, \ldots, K$ se aprenden $\mathbf{W}^{(k)}$ y $\mathbf{a}^{(k)}$, produciendo coeficientes $\alpha_{ij}^{(k)}$ y salidas $\mathbf{h}_i'^{(k)}$. En capas intermedias, se concatena:

$$\mathbf{h}_i' = \Big\|_{k=1}^{K} \sigma\left(\sum_{j\in\mathcal{N}(i)\cup\{i\}} \alpha_{ij}^{(k)}\, \mathbf{W}^{(k)}\mathbf{h}_j\right) \in \mathbb{R}^{Kd'}. \tag{31}$$

En la capa final de predicción, en lugar de concatenar se suele promediar (y aplicar la no linealidad final después):

$$\mathbf{h}_i' = \sigma\left(\frac{1}{K}\sum_{k=1}^{K}\sum_{j\in\mathcal{N}(i)\cup\{i\}} \alpha_{ij}^{(k)}\, \mathbf{W}^{(k)}\mathbf{h}_j\right). \tag{32}$$

7.3.13. Complejidad computacional

Una cabeza que calcula d' características tiene complejidad (por pasada) del orden

$$\mathcal{O}\big(|V|\, d\, d' + |E|\, d'\big),$$

pues: (i) la proyección $\mathbf{z}_i = \mathbf{W}\mathbf{h}_i$ cuesta $\mathcal{O}(|V|dd')$, y (ii) la evaluación de (28) y la agregación (30) es lineal en el número de aristas (más autolazos) [33]. Con K cabezas se multiplica el coste y memoria por un factor $\approx K$, si bien las cabezas son paralelizables.

7.3.14. Regularización y detalles prácticos

En problemas con pocos nodos etiquetados, se emplean estrategias como:

- *Dropout* sobre las entradas de capa y, de forma característica en GAT, sobre los coeficientes α_{ij}, lo que equivale a muestrear estocásticamente el vecindario durante el entrenamiento [33].
- Penalización L^2 (weight decay) y *early stopping*.
- Conexiones residuales y normalizaciones (según tarea/dataset).

7.3.15. Limitación expresiva: atención estática en GAT

Aunque conceptualmente la atención pretende depender del nodo consulta i, en la práctica la forma concreta (28) impone una restricción importante. La siguiente sección formaliza esta idea siguiendo [34].

Definición 7.2 (Atención estática)**.** *Sea $K = \{\mathbf{k}_1, \dots, \mathbf{k}_n\} \subset \mathbb{R}^d$ un conjunto de* keys *y $Q = \{\mathbf{q}_1, \dots, \mathbf{q}_m\} \subset \mathbb{R}^d$ un conjunto de* queries*. Una familia de funciones de puntuación $\mathcal{F} \subset \{f : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}\}$ tiene* puntuación estática *(para K, Q) si, para todo $f \in \mathcal{F}$, existe un índice $j_f \in \{1, \dots, n\}$ tal que para todo $i \in \{1, \dots, m\}$ y todo j,*

$$f(\mathbf{q}_i, \mathbf{k}_{j_f}) \geq f(\mathbf{q}_i, \mathbf{k}_j).$$

Diremos que una familia de atención *es estática si la atención se obtiene aplicando a f una normalización monótona (p.ej. softmax).*

Definición 7.3 (Atención dinámica)**.** *Con la notación anterior, una familia $\mathcal{F}$ tiene* puntuación dinámica *(para K, Q) si para cualquier aplicación $\varphi : \{1, \dots, m\} \to \{1, \dots, n\}$ existe $f \in \mathcal{F}$ tal que para todo i y todo $j \neq \varphi(i)$,*

$$f(\mathbf{q}_i, \mathbf{k}_{\varphi(i)}) > f(\mathbf{q}_i, \mathbf{k}_j).$$

Equivalentemente: la clave mejor puntuada puede depender esencialmente de la consulta.

Teorema 7.3 (GAT induce atención estática)**.** *Una capa GAT con puntuación* (28) *(y softmax* (29)*) sólo puede computar atención estática para $K = Q = \{\mathbf{h}_1, \dots, \mathbf{h}_n\}$. En particular, para $n > 1$ no puede realizar atención dinámica en el sentido de la Definición 7.3.*

Idea de la demostración. Escribiendo $a = [a_1 \| a_2]$, con $a_1, a_2 \in \mathbb{R}^{d'}$, la puntuación de GAT puede reescribirse como

$$a^\top [Wh_i \| Wh_j] = a_1^\top Wh_i + a_2^\top Wh_j.$$

Para un nodo i fijo, sea

$$S_i = \mathcal{N}(i) \cup \{i\}$$

el conjunto candidato de vecinos sobre el que se normaliza la atención. Al comparar dos candidatos $j, j' \in S_i$, el término $a_1^\top Wh_i$ es constante respecto de j. Como LeakyReLU y softmax son funciones monótonas crecientes, el orden relativo de las puntuaciones dentro de S_i queda determinado por

$$a_2^\top Wh_j,$$

que no depende de la representación del nodo consulta h_i. En consecuencia, para cada conjunto candidato S_i, el vecino mejor puntuado es

$$j_i^\star \in \arg\max_{j \in S_i} a_2^\top Wh_j,$$

y dicho ranking no está verdaderamente condicionado por la consulta, salvo por la restricción de qué nodos pertenecen al vecindario de i. Esta es la razón por la que GAT induce atención estática.[34]. □

Observación 7.2. El Teorema 7.3 no implica que GAT sea inútil: en muchos datasets clásicos la existencia de un ranking global de "vecinos influyentes" puede ser suficiente. Sin embargo, sí muestra que la *capacidad de selección condicionada por la consulta* es limitada.

7.3.16. GATv2: una corrección para obtener atención dinámica

Para evitar que las capas lineales se colapsen en una sola transformación, se propone cambiar el orden de operaciones y aplicar una MLP (con activación no polinómica) sobre el par (i, j). En GATv2 [34], la puntuación toma la forma:

$$e_{ij} = \mathbf{a}^\top \operatorname{LeakyReLU}(\mathbf{W}[\mathbf{h}_i \| \mathbf{h}_j]), \tag{33}$$

(posiblemente con sesgo antes de la no linealidad). Los coeficientes α_{ij} y la actualización (30) se mantienen (cambiando la forma de e_{ij}).

Teorema 7.4 (GATv2 permite atención dinámica)**.** *Una capa GATv2 con puntuación* (33) *puede computar atención dinámica (Definición 7.3) para* $K = Q = \{\mathbf{h}_1, \ldots, \mathbf{h}_n\}$.

Idea de la demostración. La puntuación (33) es esencialmente una red feedforward sobre el vector concatenado $[\mathbf{h}_i \| \mathbf{h}_j]$. Por teoremas de aproximación universal, dicha red puede aproximar funciones continuas que separen pares $(i, \varphi(i))$ del resto (i, j), permitiendo imponer que el máximo dependa de i. Aplicando la monotonía del softmax se obtiene la selección dinámica [34]. □

7.3.17. Extensión a atributos de arista (comentario)

Aunque el GAT original está formulado sólo con atributos de nodos, es natural incorporar atributos de arista $\mathbf{e}_{ij}$ ampliando la puntuación a una función $e_{ij} = \psi(\mathbf{h}_i, \mathbf{h}_j, \mathbf{e}_{ij})$ (por ejemplo concatenando una proyección de $\mathbf{e}_{ij}$). Este tipo de extensiones aparecen en implementaciones modernas (p.ej. parametrizando una dimensión de arista y el tratamiento de autolazos en librerías de GNN), y resultan útiles cuando la semántica de la ruta depende de propiedades de las conexiones.

7.3.18. Lectura (readout) para clasificación a nivel de grafo

Si la tarea es clasificación de *rutas/grafos* (no de nodos), tras L capas se obtiene $\{\mathbf{h}_i^{(L)}\}_{i \in V}$ y se aplica una función de lectura invariante READOUT (suma/media/max, o atención global) para producir un embedding de grafo:

$$\mathbf{h}_G = \operatorname{READOUT}\big(\{\mathbf{h}_i^{(L)} : i \in V\}\big),$$

que alimenta a un clasificador final.

7.4. Gated Graph Neural Network (GGNN)

7.4.1. Planteamiento: clasificación de grafos en el paradigma $f(G(N(x), V(x))) = y_{\text{class}}$

Sea un grafo dirigido o no dirigido $G = (V, E)$, con $|V| = n$. Asociamos a cada nodo $i \in V$ un vector de atributos (o *anotación*) $\mathbf{x}_i \in \mathbb{R}^F$. En la variante más general, las aristas pueden portar un tipo/etiqueta $r \in \mathcal{R}$ y/o pesos $e_{ji} \in \mathbb{R}$ para cada $(j, i) \in E$.

La tarea de *clasificación a nivel de grafo* consiste en definir una función aprendible

$$f : \big(G, \{\mathbf{x}_i\}_{i\in V}, \{e_{ji}\}_{(j,i)\in E}\big) \longmapsto y_{\text{class}} \in \{1, \dots, C\},$$

que sea invariante a permutaciones en el orden de nodos/aristas, y que integre información local y global. GGNN implementa f como: (i) un *proceso recurrente de message passing* durante L pasos, y (ii) una *lectura* (*readout*) invariante sobre el multiconjunto de estados de nodos, seguida de un clasificador.

7.4.2. Estados ocultos y padding inicial

Fijamos una dimensión de estado d (en PyG, `out_channels`). Definimos estados ocultos $\mathbf{h}_i^{(t)} \in \mathbb{R}^d$ para $t = 0, 1, \dots, L$. En GGNN, el estado inicial se obtiene embebiendo $\mathbf{x}_i$ en $\mathbb{R}^d$. Una elección estándar (y la que implementa `GatedGraphConv` cuando $F \le d$) es

$$\mathbf{h}_i^{(0)} = \mathbf{x}_i \,\|\, \mathbf{0} \in \mathbb{R}^d, \tag{34}$$

donde $\|$ denota concatenación y se rellenan con ceros las $d - F$ últimas coordenadas.

7.4.3. Message Passing recurrente: mensajes y agregación

Sea $\mathcal{N}(i) = \{j \in V : (j, i) \in E\}$ el vecindario entrante de i. En el paso $t \mapsto t + 1$, GGNN computa primero un mensaje agregado $\mathbf{m}_i^{(t+1)}$ a partir de los estados vecinos $\{\mathbf{h}_j^{(t)}\}_{j\in\mathcal{N}(i)}$. La forma base deldel artículo original de GGNN [39]es lineal por arista y suma por vecindario:

$$\mathbf{m}_i^{(t+1)} = \sum_{j\in\mathcal{N}(i)} \mathbf{M}_{j\to i}^{(t)}\big(\mathbf{h}_j^{(t)}\big), \qquad \mathbf{M}_{j\to i}^{(t)}(\mathbf{h}) = e_{ji}\, \mathbf{\Theta}^{(t)}\mathbf{h}, \tag{35}$$

donde $\mathbf{\Theta}^{(t)} \in \mathbb{R}^{d\times d}$ es una matriz aprendible (posiblemente distinta por paso) y e_{ji} es un peso escalar de arista (si no existe, se toma $e_{ji} = 1$).

Generalización con tipos de arista (formulación original). En muchos problemas, las aristas tienen una etiqueta discreta $r \in \mathcal{R}$ (p.ej. tipo de relación). Entonces se define un conjunto de matrices $\{\mathbf{\Theta}_r^{(t)}\}_{r\in\mathcal{R}}$ y

$$\mathbf{m}_i^{(t+1)} = \sum_{r\in\mathcal{R}} \sum_{j\in\mathcal{N}_r(i)} \mathbf{\Theta}_r^{(t)}\, \mathbf{h}_j^{(t)}, \tag{36}$$

donde $\mathcal{N}_r(i) = \{j : (j,i) \in E \text{ con tipo } r\}$. Esta variante captura *heterogeneidad relacional*.

Observación 7.3 (Invarianza a permutaciones locales). Las expresiones (35)–(36) dependen del vecindario únicamente a través de una suma sobre $\mathcal{N}(i)$, por lo que son invariantes a cualquier reordenación de vecinos.

7.4.4. Actualización con compuertas: GRU sobre grafos

La idea fundamental de GGNN es que la actualización del estado del nodo se realiza con una celda recurrente con compuertas (*Gated Recurrent Unit*, GRU). Dado el mensaje $\mathbf{m}_i^{(t+1)}$ y el estado previo $\mathbf{h}_i^{(t)}$, se define:

$$\mathbf{h}_i^{(t+1)} = \mathrm{GRU}\Big(\mathbf{m}_i^{(t+1)},\, \mathbf{h}_i^{(t)}\Big). \tag{37}$$

Para explicitar (37), una celda GRU con dimensión d se define por:

$$\mathbf{z}_i^{(t+1)} = \sigma\Big(\mathbf{W}_z\, \mathbf{m}_i^{(t+1)} + \mathbf{U}_z\, \mathbf{h}_i^{(t)} + \mathbf{b}_z\Big), \tag{38}$$

$$\mathbf{r}_i^{(t+1)} = \sigma\Big(\mathbf{W}_r\, \mathbf{m}_i^{(t+1)} + \mathbf{U}_r\, \mathbf{h}_i^{(t)} + \mathbf{b}_r\Big), \tag{39}$$

$$\widetilde{\mathbf{h}}_i^{(t+1)} = \tanh\Big(\mathbf{W}_h\, \mathbf{m}_i^{(t+1)} + \mathbf{U}_h\, (\mathbf{r}_i^{(t+1)} \odot \mathbf{h}_i^{(t)}) + \mathbf{b}_h\Big), \tag{40}$$

$$\mathbf{h}_i^{(t+1)} = (\mathbf{1} - \mathbf{z}_i^{(t+1)}) \odot \mathbf{h}_i^{(t)} + \mathbf{z}_i^{(t+1)} \odot \widetilde{\mathbf{h}}_i^{(t+1)}. \tag{41}$$

Aquí σ es la sigmoide, $\odot$ denota producto elemento a elemento y $\mathbf{W}_\bullet, \mathbf{U}_\bullet \in \mathbb{R}^{d\times d}$, $\mathbf{b}_\bullet \in \mathbb{R}^d$ son parámetros aprendibles. Las compuertas $\mathbf{z}$ y $\mathbf{r}$ controlan (respectivamente) cuánto se actualiza el estado y cuánto del pasado se "resetea", mitigando problemas de gradientes y permitiendo message passing profundo en t.

7.4.5. Forma matricial

Apilemos estados en una matriz $\mathbf{H}^{(t)} \in \mathbb{R}^{n\times d}$, donde la fila i es $\mathbf{h}_i^{(t)\top}$. Sea $\mathbf{A}$ la adyacencia dirigida (sin normalizar) y $\mathbf{E}$ una matriz de pesos de arista tal que $(\mathbf{E})_{ji} = e_{ji}$ si $(j,i) \in E$, y 0 en otro caso. Entonces la agregación (35) puede escribirse como:

$$\mathbf{M}^{(t+1)} = (\mathbf{E} \odot \mathbf{A})^\top\, \mathbf{H}^{(t)}\, (\Theta^{(t)})^\top, \tag{42}$$

donde $\mathbf{M}^{(t+1)} \in \mathbb{R}^{n\times d}$ apila $\mathbf{m}_i^{(t+1)}$ por filas. La actualización GRU se aplica fila a fila:

$$\mathbf{H}^{(t+1)} = \mathrm{GRUCell}\big(\mathbf{M}^{(t+1)},\, \mathbf{H}^{(t)}\big).$$

Complejidad. Cada paso t cuesta:

$$\mathcal{O}\big(|E|\, d + n\, d^2\big),$$

donde $|E|d$ proviene de la propagación dispersa y nd^2 de las multiplicaciones densas asociadas a la GRU. El coste total escala linealmente con el número de pasos L: $\mathcal{O}\big(L(|E|d + nd^2)\big)$.

7.4.6. Relación con otros paradigmas: sistema dinámico y MPNN

GGNN puede interpretarse como un *sistema dinámico discreto* sobre el grafo:

$$\mathbf{H}^{(t+1)} = \Phi_\theta\big(\mathbf{H}^{(t)}; G\big), \qquad t = 0, \dots, L-1,$$

donde Φ_θ combina una agregación lineal sobre el grafo y una actualización no lineal con memoria (GRU). Esta formulación coincide con el marco MPNN: el mensaje es $\mathbf{M}_{j\to i}^{(t)}$ y la función de actualización es la GRU. A diferencia de GCN/GIN, que usan actualizaciones feedforward sin memoria explícita en t, GGNN incorpora *estado recurrente*, lo que a menudo es útil cuando se desea que el número de pasos L tenga interpretación semántica (p.ej. alcance L-saltos o iteraciones de un algoritmo sobre grafos).

7.4.7. Readout para clasificación de grafos

Para clasificación a nivel de grafo, tras L pasos se dispone de $\{\mathbf{h}_i^{(L)}\}_{i\in V}$. Se requiere una lectura invariante a permutaciones READOUT para construir un embedding global $\mathbf{h}_G$.

Readout tipo GGNN (con compuerta). En [39] se propone una lectura que combina estado final y anotación inicial:

$$\mathbf{g}_i = \sigma\Big(\mathbf{W}_g\,[\mathbf{h}_i^{(L)} \| \mathbf{x}_i] + \mathbf{b}_g\Big), \tag{43}$$

$$\mathbf{o}_i = \tanh\Big(\mathbf{W}_o\,\mathbf{h}_i^{(L)} + \mathbf{b}_o\Big), \tag{44}$$

$$\mathbf{h}_G = \sum_{i\in V} \mathbf{g}_i \odot \mathbf{o}_i. \tag{45}$$

La suma asegura invariancia a permutaciones. Intuitivamente, $\mathbf{g}_i$ decide qué nodos contribuyen más al embedding global en cada coordenada.

Finalmente, un clasificador (p.ej. una MLP) produce probabilidades de clase:

$$\widehat{\mathbf{y}}_G = \mathrm{softmax}\big(\mathrm{MLP}(\mathbf{h}_G)\big).$$

7.4.8. Correspondencia con `GatedGraphConv` de PyTorch Geometric

La capa `GatedGraphConv(out_channels=d, num_layers=L)` de PyTorch Geometric sigue este esquema recurrente con GRU bajo las convenciones de la librería:

$$h_i^{(0)} = x_i \| 0,$$

cuando la dimensión de entrada es menor que `out_channels`, y

$$m_i^{(t+1)} = \sum_{j\in\mathcal{N}(i)} e_{ji}\Theta^{(t)} h_j^{(t)},$$

donde e_{ji} puede proporcionarse mediante `edge_weight`. La actualización se realiza mediante

$$h_i^{(t+1)} = \mathrm{GRU}\left(m_i^{(t+1)}, h_i^{(t)}\right), \qquad t = 0, \ldots, L-1.$$

Esta implementación contempla pesos escalares de arista y agregación configurable. Si se desean tipos discretos de arista con matrices distintas por relación, conviene emplear una extensión relacional, por ejemplo R-GCN, o codificar explícitamente los tipos de relación.

8. Experimentación

8.1. Dataset: MUTAG

El dataset MUTAG es un conjunto clásico de clasificación de grafos moleculares. Está formado por 188 compuestos químicos, representados como grafos, y cada grafo corresponde a una molécula. Desde el punto de vista químico, los vértices del grafo representan átomos y las aristas representan enlaces químicos entre esos átomos. Los átomos de hidrógeno explícitos no se incluyen en esta representación, por lo que el grafo describe la estructura principal de la molécula a partir de los átomos distintos del hidrógeno.

El objetivo de clasificación consiste en predecir si un compuesto tiene o no efecto mutagénico sobre una bacteria. Por tanto, cada molécula tiene una etiqueta global $y \in \{1, -1\}$. En esta codificación, la etiqueta $y = 1$ (que representa el 66,5 % del dataset) se interpreta como la clase positiva, asociada a un compuesto mutagénico, mientras que $y = -1$ (el 33,5 % restante) representa la clase negativa, asociada a un compuesto no mutagénico. Esta es una tarea de clasificación de grafos, ya que la etiqueta no se asigna a cada átomo individual, sino a la molécula completa.

El dataset de trabajo se encuentra separado en distintos archivos donde cada archivo contiene un aspecto informativo del grafo de manera que al agregar la información se pueden reconstruir estos grafos moleculares. El archivo `MUTAG_graph_labels.txt` contiene una etiqueta por molécula; en este caso hay 188 líneas, una para cada grafo. El archivo `MUTAG_node_labels.txt` contiene una etiqueta por átomo; en esta versión hay 3371 átomos en total. Cada etiqueta de nodo codifica el tipo de elemento químico según la correspondencia

Etiqueta	0	1	2	3	4	5	6
Átomo	C	N	O	F	I	Cl	Br

Por otro lado, el archivo `MUTAG_edge_labels.txt` contiene una etiqueta para cada enlace almacenado en la matriz de adyacencia. Las etiquetas de arista indican el tipo de enlace químico:

Etiqueta	0	1	2	3
Tipo de enlace	aromático	simple	doble	triple

En el formato habitual de MUTAG, la conectividad se almacena en el archivo `MUTAG_A.txt`, que contiene pares de identificadores de nodos. Además, `MUTAG_graph_indicator.txt` indica a qué molécula pertenece cada átomo. De esta forma, la combinación de estos archivos permite reconstruir cada molécula como un grafo etiquetado: los nodos tienen etiquetas químicas, las aristas tienen tipos de enlace y cada grafo completo tiene una etiqueta de mutagenicidad.

De manera ilustrativa, la primera molécula del dataset tiene etiqueta global $y = 1$, por lo que pertenece a la clase mutagénica. En esta molécula tiene en `MUTAG_node_labels.txt` 17 átomos: 14 átomos de carbono, un átomo de nitrógeno y dos átomos de oxígeno; mientras en en `MUTAG_edge_labels.txt` se almacena la relación (el tipo de enlace) entre los distintos nodos (átomos). Esto nos permite reconstruir la siguiente figura, donde las aristas discontinuas indican enlaces aromáticos, la arista continua indica un enlace simple y la arista doble indica un enlace doble.

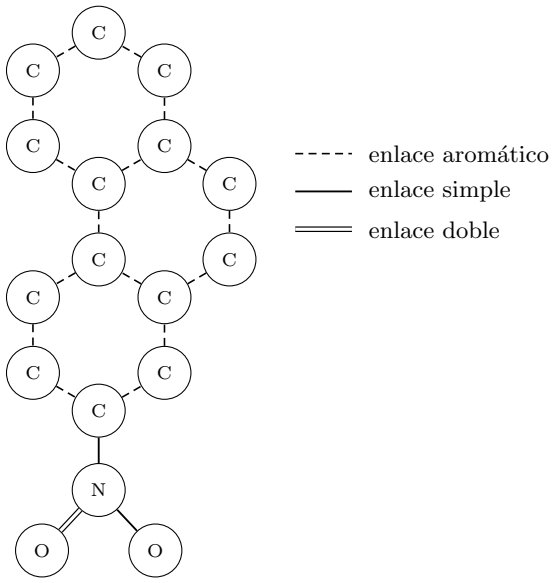

Figura 9: Representación de la primera molécula del dataset MUTAG. La etiqueta global $y = 1$ indica que la molécula pertenece a la clase mutagénica. Elaboración propia.

Una vez reconstruido el grafo molecular, la información no se introduce en la red neuronal como una figura química, sino como un conjunto de tensores. En PyTorch Geometric, cada molécula se representa mediante un objeto de datos de la forma

$$\texttt{Data}(x, \texttt{edge_index}, \texttt{edge_attr}, y),$$

donde x contiene los atributos de los nodos, `edge_index` codifica la conectividad del grafo, `edge_attr` contiene los atributos de las aristas y y es la etiqueta global

de la molécula. Para la primera molécula de MUTAG se tiene

$$|V| = 17, \qquad |\mathcal{E}_{\text{dir}}| = 38, \qquad y = 1.$$

Aquí $|\mathcal{E}_{\text{dir}}| = 38$ porque el fichero `MUTAG_A.txt` almacena cada enlace en ambos sentidos. Es decir, si existe un enlace químico no dirigido entre dos átomos u y v, en la entrada aparecen las dos aristas dirigidas $u \to v$ y $v \to u$. Por tanto, los 19 enlaces químicos de la molécula se convierten en 38 columnas en `edge_index`. Las etiquetas atómicas de esta molécula son

$$\ell_V = (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 2, 2),$$

donde 0 representa carbono, 1 nitrógeno y 2 oxígeno. Como la red neuronal opera con vectores numéricos, estas etiquetas discretas se codifican mediante one-hot. Usando el orden

$$(C, N, O, F, I, Cl, Br),$$

la matriz de atributos de nodo es

$$x = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{pmatrix} \in \mathbb{R}^{17 \times 7}.$$

Cada fila de x representa un átomo. Por ejemplo, las primeras catorce filas corresponden a átomos de carbono, la fila 15 corresponde al nitrógeno y las filas 16 y 17 corresponden a oxígenos. La conectividad se almacena en formato disperso mediante el tensor `edge_index`. Para esta molécula, usando indexación local desde cero, se obtiene

$$\texttt{edge_index} = \left[\begin{array}{cccccccccc|c|cccccccc} 1 & 0 & 2 & 1 & 3 & 2 & 4 & 3 & 5 & 4 & \cdots & 13 & 8 & 14 & 12 & 15 & 14 & 16 & 14 \\ 0 & 1 & 1 & 2 & 2 & 3 & 3 & 4 & 4 & 5 & & 8 & 13 & 12 & 14 & 14 & 15 & 14 & 16 \end{array}\right].$$

Cada columna representa una arista dirigida. Por ejemplo, las dos primeras columnas indican las conexiones

$$1 \to 0, \qquad 0 \to 1,$$

que corresponden al mismo enlace químico almacenado en ambos sentidos. Las etiquetas de arista asociadas a esas 38 aristas dirigidas son

$$\ell_E = (\underbrace{0, \ldots, 0}_{32 \text{ aristas aromáticas}}, 1, 1, 2, 2, 1, 1).$$

Usando el orden

$$(\text{aromático}, \text{simple}, \text{doble}, \text{triple}),$$

cada etiqueta de arista se codifica también en formato one-hot:

$$0 \mapsto (1,0,0,0), \qquad 1 \mapsto (0,1,0,0), \qquad 2 \mapsto (0,0,1,0), \qquad 3 \mapsto (0,0,0,1).$$

Por tanto,

$$\texttt{edge_attr} = \begin{pmatrix} \mathbf{1}_{32} \otimes (1,0,0,0) \\ (0,1,0,0) \\ (0,1,0,0) \\ (0,0,1,0) \\ (0,0,1,0) \\ (0,1,0,0) \\ (0,1,0,0) \end{pmatrix} \in \mathbb{R}^{38 \times 4}.$$

Esta matriz contiene una fila por cada columna de `edge_index`. Así, la fila r de `edge_attr` describe el tipo de enlace correspondiente a la arista dirigida almacenada en la columna r de `edge_index`. De forma equivalente, si se eliminan las repeticiones debidas a la doble dirección, los 19 enlaces químicos no dirigidos de la primera molécula pueden resumirse como

Enlace no dirigido	Tipo de enlace
$(0,1)$	aromático
$(1,2)$	aromático
$(2,3)$	aromático
$(3,4)$	aromático
$(4,5)$	aromático
$(0,5)$	aromático
$(4,6)$	aromático
$(6,7)$	aromático
$(7,8)$	aromático
$(8,9)$	aromático
$(3,9)$	aromático
$(9,10)$	aromático
$(10,11)$	aromático
$(11,12)$	aromático
$(12,13)$	aromático
$(8,13)$	aromático
$(12,14)$	simple
$(14,15)$	doble
$(14,16)$	simple

Esta tabla es una forma más interpretable de ver la misma información que aparece en `edge_index` y `edge_attr`. En consecuencia, la entrada de la primera molécula al modelo queda representada como:

$$G_1 = \left(x \in \mathbb{R}^{17\times 7}, \texttt{edge_index} \in \mathbb{N}^{2\times 38}, \texttt{edge_attr} \in \mathbb{R}^{38\times 4}, y = 1\right).$$

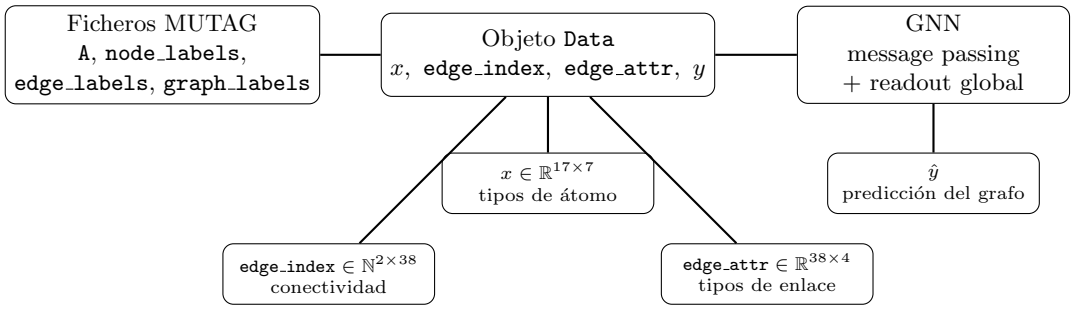

Figura 10: Transformación de los ficheros de MUTAG en tensores de entrada para una red neuronal de grafos. Elaboración propia.

La red no recibe directamente la etiqueta y como entrada predictiva; esta se utiliza como objetivo supervisado durante el entrenamiento. La información que sí recibe el modelo es la estructura molecular y sus atributos: tipos de átomos, conectividad y, en los modelos que lo permiten, tipos de enlace.

En una capa de message passing, la representación inicial de cada átomo es su vector one-hot:

$$h_v^{(0)} = x_v.$$

A partir de ahí, cada nodo actualiza su representación agregando información de sus vecinos. De forma general, una capa puede escribirse como

$$m_v^{(k)} = \mathrm{AGG}^{(k)}\left(\left\{M^{(k)}\left(h_v^{(k)}, h_u^{(k)}, e_{uv}\right) : u \in \mathcal{N}(v)\right\}\right),$$

$$h_v^{(k+1)} = U^{(k)}\left(h_v^{(k)}, m_v^{(k)}\right),$$

donde $h_v^{(k)}$ es la representación del átomo v en la capa k, e_{uv} representa el tipo de enlace entre u y v, $M^{(k)}$ es la función de mensaje, $\mathrm{AGG}^{(k)}$ es una agregación invariante al orden y $U^{(k)}$ es la función de actualización.

Sin embargo es necesario destacar que no todas las arquitecturas utilizarán toda la información disponible. Por ejemplo, modelos como GCN, GraphSAGE o GIN pueden usar x y `edge_index`, pero no necesariamente `edge_attr`. En ese caso, el modelo conoce qué átomos están conectados, pero no distingue si el enlace es aromático, simple o doble. En cambio, modelos como GINE incorporan explícitamente `edge_attr` en el cálculo del mensaje. Para MUTAG esto es especialmente relevante, ya que el tipo de enlace químico forma parte de la estructura molecular. Tras varias capas, cada nodo tiene una representación

aprendida $h_v^{(K)}$. Como la tarea es de clasificación de grafos, estas representaciones de átomos deben resumirse en un único vector de molécula:

$$h_G = \text{READOUT}\left(\left\{h_v^{(K)} : v \in V\right\}\right).$$

Una elección habitual es la media global:

$$h_G = \frac{1}{|V|}\sum_{v \in V} h_v^{(K)}.$$

Finalmente, este vector de grafo se pasa por un clasificador, normalmente una MLP, para producir la predicción

$$\hat{y} = \text{MLP}(h_G).$$

En la primera molécula, el objetivo de entrenamiento asociado es $y = 1$, por lo que el modelo ajusta sus parámetros para que la predicción $\hat{y}$ se acerque a la clase mutagénica.

8.2. Protocolo de experimentación

El objetivo de la experimentación es comparar distintas arquitecturas neuronales para clasificación de grafos en el dataset MUTAG bajo un protocolo común y reproducible. Dado que MUTAG es un conjunto de datos pequeño, formado por 188 grafos moleculares, se utiliza validación cruzada estratificada con varias semillas aleatorias. De esta forma, la estimación del rendimiento no depende de una única partición entrenamiento–test y se reduce la variabilidad asociada al reparto de las moléculas entre los distintos subconjuntos.

Para cada semilla $s \in \{0, 1, 2, 3, 4\}$, se aplica una validación cruzada estratificada de $K = 10$ pliegues. La estratificación se realiza con respecto a la etiqueta global de cada grafo, de modo que cada pliegue conserva aproximadamente la proporción de moléculas mutagénicas y no mutagénicas del conjunto completo. En cada iteración de la validación cruzada, uno de los pliegues se reserva como conjunto de test y los nueve pliegues restantes forman un conjunto $\mathcal{D}_{\text{train_val}}$. Este conjunto se divide de nuevo, también de forma estratificada, en entrenamiento y validación, reservando un 15 % para validación:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}.$$

El conjunto de test permanece completamente separado durante el entrenamiento y no se utiliza ni para seleccionar hiperparámetros ni para decidir el criterio de parada. Su único uso es la evaluación final del modelo seleccionado en cada pliegue.

Todos los modelos reciben como entrada la representación tensorial de cada molécula,

$$G = (x, \texttt{edge_index}, \texttt{edge_attr}, y),$$

donde x contiene los tipos de átomo codificados mediante one-hot, `edge_index` almacena la conectividad del grafo y `edge_attr` contiene los tipos de enlace,

también codificados en formato one-hot. No obstante, no todas las arquitecturas emplean la misma información. Los modelos que no incorporan atributos de arista utilizan únicamente x y `edge_index`, mientras que los modelos *edge-aware* incorporan explícitamente `edge_attr` en el cálculo de los mensajes.

La comparación se organiza en tres grupos de modelos. En primer lugar, se incluye un baseline neuronal sin *message passing*, denotado como `MLP+pool`, que aplica una red MLP sobre las características de los nodos y obtiene una representación global del grafo mediante una operación de *readout*. En segundo lugar, se evalúan arquitecturas clásicas de redes neuronales de grafos: GCN, GraphSAGE, GIN, GINE, GAT, GATv2 y GGNN. Por último, se consideran variantes que incorporan explícitamente atributos de arista, como GAT-edge y GATv2-edge, además de un modelo híbrido exploratorio GINE+GATv2. Este último aplica primero capas GINE para integrar la información de los enlaces químicos y posteriormente una capa GATv2 con atributos de arista para ponderar dinámicamente la información procedente de los vecinos.

En todos los modelos neuronales se emplea la misma configuración base de entrenamiento. Las redes se entrenan durante un máximo de 250 épocas, con *early stopping* de paciencia 40 (es decir, el entrenamiento se detiene antes si tras mejorar en una epoch, no mejora en las siguientes 40). El criterio monitorizado para seleccionar el mejor checkpoint es la *balanced accuracy* en el conjunto de validación. La función de pérdida utilizada es la entropía cruzada, adecuada para la clasificación binaria formulada con dos clases. La optimización se realiza con AdamW, usando tasa de aprendizaje 0,003 y regularización *weight decay* = 10^{-4}. El tamaño de batch es 32, la dimensión oculta es 64, el número de capas es 3, el dropout es 0.2 y el *readout* global se realiza mediante media:

$$h_G = \frac{1}{|V|} \sum_{v \in V} h_v^{(K)}.$$

En los modelos basados en atención se utilizan 4 cabezas de atención.

Para cada combinación de semilla, pliegue y modelo, el entrenamiento produce una historia de pérdidas y métricas en entrenamiento, validación y test. Sin embargo, la pérdida y las métricas de test registradas por época se utilizan únicamente con fines de visualización y análisis posterior de las curvas de aprendizaje. En particular, el conjunto de test no interviene en el proceso de selección del modelo. El checkpoint final de cada pliegue corresponde siempre a la época con mejor resultado en validación según la métrica monitorizada.

Finalmente, una vez entrenados todos los modelos en todos los pliegues y semillas, se almacenan los resultados individuales por pliegue y se calculan agregados por modelo y semilla. Las métricas globales se obtendrán promediando los resultados de test sobre los pliegues y posteriormente resumiendo el comportamiento entre las distintas semillas. Esta agregación permite comparar los modelos no solo por su rendimiento medio, sino también por la estabilidad de sus resultados frente a las particiones aleatorias del dataset.

8.3. Resultados

En esta sección se presentan los resultados obtenidos en MUTAG para los modelos considerados en el experimento. Para no sobrecargar la lectura, los resultados se organizan en tres bloques, uno por métrica: exactitud (*accuracy*), *recall* macro -equivalente en este contexto a la *balanced accuracy*- y F1 macro. En cada bloque se agrupan dos tablas en una misma unidad: la primera recoge la media de test obtenida por cada modelo y semilla, y la segunda recoge la desviación típica entre pliegues para esa misma semilla. Todas las magnitudes se expresan en porcentaje. Dado que MUTAG es un problema de clasificación binaria, se fija una clase positiva y una clase negativa. En este trabajo se toma como clase positiva la clase mutagénica y como clase negativa la clase no mutagénica. A partir de esta elección, se denotan por (TP) los verdaderos positivos, por (TN) los verdaderos negativos, por (FP) los falsos positivos y por (FN) los falsos negativos. Por tanto, el número total de ejemplos evaluados es [N = TP + TN + FP + FN.] Con esta notación, las métricas empleadas se definen como sigue:

- **Exactitud (*accuracy*).** Mide la proporción total de ejemplos correctamente clasificados, tanto positivos como negativos:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

- ***Recall* macro.** El *recall* mide la proporción de ejemplos de una clase que han sido correctamente identificados. Para la clase positiva se tiene

$$\text{Recall}_{+} = \frac{TP}{TP + FN},$$

mientras que, para la clase negativa, el *recall* viene dado por

$$\text{Recall}_{-} = \frac{TN}{TN + FP}.$$

El *recall* macro se obtiene promediando ambos valores:

$$\text{Recall}_{macro} = \frac{1}{2}\left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP}\right).$$

En un problema binario, esta expresión coincide con la *balanced accuracy*, ya que calcula la media del *recall* de la clase positiva y el *recall* de la clase negativa.

- **F1 macro.** Para definir el F1 de cada clase se introduce primero la precisión. Para la clase positiva,

$$\text{Precision}_{+} = \frac{TP}{TP + FP},$$

y para la clase negativa,

$$\text{Precision}_- = \frac{TN}{TN + FN}.$$

A partir de precisión y *recall*, el F1 de la clase positiva es

$$\text{F1}_+ = \frac{2\,\text{Precision}_+\,\text{Recall}_+}{\text{Precision}_+ + \text{Recall}_+},$$

y el F1 de la clase negativa es

$$\text{F1}_- = \frac{2\,\text{Precision}_-\,\text{Recall}_-}{\text{Precision}_- + \text{Recall}_-}.$$

Finalmente, el F1 macro se calcula como la media aritmética del F1 de ambas clases:

$$\text{F1}_{macro} = \frac{1}{2}\left(\text{F1}_+ + \text{F1}_-\right).$$

Cada celda procede del resumen por semilla de una validación cruzada estratificada de 10 pliegues. El protocolo separa el conjunto de test de cada pliegue de la selección de hiperparámetros y del early stopping, que se realizan usando validación interna; por tanto, las métricas de test se interpretan como evaluación final del modelo seleccionado en cada pliegue.

Exactitud. La Tabla 1 muestra la exactitud media y su desviación típica. El modelo híbrido `GINE+GATv2` alcanza la mayor exactitud media global, con un promedio del 83.92 % entre semillas. Además, obtiene el mejor valor en cuatro de las cinco semillas; la excepción es la seed 0, donde `GINE` alcanza el máximo. Tras el híbrido aparecen `GINE`, `GIN` y `GGNN`, con valores medios en torno al 78.9–79.9 %. En cambio, el baseline `MLP+pool`, que no realiza message passing, queda claramente por debajo, con una media global del 70.51 %.

Cuadro 1: Exactitud de test por modelo y semilla en MUTAG. El panel superior muestra la media y el panel inferior la desviación típica entre pliegues. Los valores están expresados en porcentaje.

(a) Media por semilla y promedio global

Modelo	Seed 0	Seed 1	Seed 2	Seed 3	Seed 4	Prom.
`MLP+pool`	70.76	71.20	71.32	68.57	70.70	70.51
`GCN`	71.29	72.28	75.03	74.91	71.67	73.04
`GraphSAGE-mean`	71.29	74.97	71.81	76.02	75.44	73.91
`GraphSAGE-pool`	72.87	74.44	74.47	74.91	73.30	74.00
`GIN`	77.63	76.14	81.37	79.18	80.12	78.89
`GINE`	**84.59**	77.63	80.26	77.13	79.74	79.87
`GAT`	73.92	75.44	72.89	77.02	73.83	74.62
`GAT-edge`	73.95	76.58	78.30	77.60	74.88	76.26
`GATv2`	72.34	72.81	72.87	74.36	72.78	73.03
`GATv2-edge`	74.50	73.39	74.44	75.44	76.55	74.87
`GGNN`	80.85	78.74	77.69	79.65	77.54	78.89
`GINE+GATv2`	82.54	**84.53**	**83.57**	**82.87**	**86.11**	**83.92**

(b) Desviación típica entre pliegues

Modelo	Seed 0	Seed 1	Seed 2	Seed 3	Seed 4	Prom.
`MLP+pool`	12.91	8.64	7.78	7.02	7.76	8.82
`GCN`	16.47	9.89	6.92	8.95	9.38	10.32
`GraphSAGE-mean`	13.83	12.59	8.97	8.49	8.11	10.40
`GraphSAGE-pool`	10.79	12.65	8.93	10.82	10.66	10.77
`GIN`	9.52	10.49	4.51	8.27	14.08	9.38
`GINE`	6.74	10.53	9.53	8.58	13.34	9.75
`GAT`	12.41	13.74	6.19	8.73	9.60	10.14
`GAT-edge`	12.65	11.90	7.89	6.71	9.83	9.80
`GATv2`	10.87	11.26	6.83	9.97	9.87	9.76
`GATv2-edge`	13.48	11.12	7.48	8.67	6.89	9.53
`GGNN`	10.36	10.78	8.45	9.72	15.00	10.86
`GINE+GATv2`	10.46	7.43	11.69	11.89	8.54	10.00

La desviación típica entre pliegues se sitúa habitualmente alrededor de 9–11 puntos porcentuales, lo que refleja la variabilidad esperable en un conjunto pequeño como MUTAG. Por ello, aunque la ordenación media favorece claramente a `GINE+GATv2`, resulta conveniente interpretar las diferencias pequeñas entre modelos cercanos con cautela.

Balanced accuracy / recall macro. La Tabla 2 recoge la balanced accuracy. Esta métrica es especialmente relevante porque compensa el peso de las clases y permite detectar si una buena exactitud se debe principalmente al comportamiento sobre la clase mayoritaria. En este caso, `GINE+GATv2` vuelve a obtener el mejor promedio global, con un 82.79 %. El segundo grupo lo forman `GIN` y `GINE`, con medias del 79.35 % y 78.75 %, respectivamente, mientras que `GGNN` queda algo por debajo con un 76.44 %. Por semillas, `GINE+GATv2` domina en las seeds

1, 3 y 4, mientras que `GINE` y `GIN` presentan los mejores valores en las seeds 0 y 2.

Cuadro 2: Balanced accuracy / recall macro de test por modelo y semilla en MUTAG. El panel superior muestra la media y el panel inferior la desviación típica entre pliegues. Los valores están expresados en porcentaje.

(a) Media por semilla y promedio global

Modelo	Seed 0	Seed 1	Seed 2	Seed 3	Seed 4	Prom.
`MLP+pool`	64.90	62.19	64.46	62.35	61.12	63.00
`GCN`	67.83	67.17	69.10	70.16	64.90	67.83
`GraphSAGE-mean`	67.45	70.40	65.44	70.52	68.40	68.44
`GraphSAGE-pool`	67.77	70.50	71.50	71.90	67.30	69.79
`GIN`	77.62	77.93	**83.04**	78.99	79.18	79.35
`GINE`	**82.43**	78.58	77.43	76.25	79.04	78.75
`GAT`	69.88	70.25	68.69	72.19	68.56	69.91
`GAT-edge`	71.43	71.77	72.73	71.80	70.20	71.59
`GATv2`	68.39	68.00	67.29	69.15	69.44	68.45
`GATv2-edge`	70.24	68.02	69.82	71.57	71.33	70.20
`GGNN`	77.79	77.02	74.28	76.68	76.42	76.44
`GINE+GATv2`	80.68	**82.99**	81.68	**82.01**	**86.57**	**82.79**

(b) Desviación típica entre pliegues

Modelo	Seed 0	Seed 1	Seed 2	Seed 3	Seed 4	Prom.
`MLP+pool`	11.74	10.16	5.86	7.86	9.31	8.99
`GCN`	16.80	9.56	7.02	11.03	7.11	10.30
`GraphSAGE-mean`	12.95	11.88	9.58	10.72	7.87	10.60
`GraphSAGE-pool`	10.53	13.31	7.73	11.68	10.38	10.73
`GIN`	8.08	9.44	4.76	8.42	14.41	9.02
`GINE`	7.12	7.97	11.76	9.66	14.23	10.15
`GAT`	12.93	16.04	5.33	9.85	9.54	10.74
`GAT-edge`	13.74	12.82	9.19	8.81	9.45	10.80
`GATv2`	11.47	11.16	7.25	11.63	7.48	9.80
`GATv2-edge`	14.89	12.02	7.08	9.41	7.51	10.18
`GGNN`	11.11	8.78	11.77	10.43	16.89	11.80
`GINE+GATv2`	13.55	8.65	12.00	12.22	8.95	11.07

La diferencia entre exactitud y balanced accuracy es apreciable en varios modelos, especialmente en el baseline `MLP+pool` y en algunas arquitecturas atencionales sin atributos de arista. Esto indica que la exactitud por sí sola puede ofrecer una visión demasiado optimista del rendimiento cuando el comportamiento por clase no está equilibrado.

F1 macro. La Tabla 3 presenta el F1 macro, que combina precisión y recall de forma simétrica entre clases. La tendencia es muy similar a la observada con balanced accuracy: `GINE+GATv2` obtiene el mejor promedio global, con un 82.00 %, seguido por `GINE`, `GIN` y `GGNN`. La proximidad entre `GINE` y `GIN` su-

giere que incorporar atributos de arista es beneficioso, aunque el efecto no es uniforme para todas las familias. En las variantes atencionales, las versiones edge-aware mejoran moderadamente a sus equivalentes sin atributos de arista, pero no alcanzan el rendimiento de los modelos basados en GIN/GINE.

Cuadro 3: F1 macro de test por modelo y semilla en MUTAG. El panel superior muestra la media y el panel inferior la desviación típica entre pliegues. Los valores están expresados en porcentaje.

(a) Media por semilla y promedio global

Modelo	Seed 0	Seed 1	Seed 2	Seed 3	Seed 4	Prom.
`MLP+pool`	64.70	60.41	64.87	61.66	60.31	62.39
`GCN`	66.74	67.55	69.81	70.19	65.34	67.93
`GraphSAGE-mean`	67.05	71.09	65.38	70.51	69.16	68.64
`GraphSAGE-pool`	67.38	70.57	71.16	70.91	67.60	69.52
`GIN`	75.90	75.06	80.18	77.45	78.38	77.40
`GINE`	**82.54**	76.04	77.28	74.80	77.76	77.69
`GAT`	69.52	69.46	68.49	72.95	68.65	69.81
`GAT-edge`	70.78	71.82	73.48	72.15	70.24	71.69
`GATv2`	67.51	68.28	67.69	69.42	69.42	68.47
`GATv2-edge`	69.93	68.38	70.29	71.40	72.02	70.40
`GGNN`	77.97	76.66	73.83	76.85	74.41	75.94
`GINE+GATv2`	79.61	**82.64**	**81.61**	**81.19**	**84.93**	**82.00**

(b) Desviación típica entre pliegues

Modelo	Seed 0	Seed 1	Seed 2	Seed 3	Seed 4	Prom.
`MLP+pool`	13.81	14.29	6.80	8.63	11.88	11.08
`GCN`	18.04	9.52	7.25	11.05	8.06	10.79
`GraphSAGE-mean`	14.56	12.99	11.15	11.23	9.05	11.80
`GraphSAGE-pool`	11.83	13.80	8.83	12.76	11.81	11.80
`GIN`	9.25	10.37	4.65	8.88	14.61	9.55
`GINE`	7.09	9.89	11.20	9.11	14.13	10.29
`GAT`	13.84	17.88	5.70	10.20	10.46	11.62
`GAT-edge`	13.98	13.81	9.61	8.99	9.53	11.18
`GATv2`	11.76	11.40	7.89	12.03	9.11	10.44
`GATv2-edge`	15.88	11.81	7.90	10.65	7.84	10.82
`GGNN`	11.50	10.19	11.17	10.44	18.73	12.41
`GINE+GATv2`	13.87	8.35	12.17	12.49	8.95	11.16

En conjunto, las tres métricas apuntan a la misma lectura: el modelo híbrido `GINE+GATv2` es el más competitivo y mantiene una ventaja consistente a través de las semillas. Los modelos `GIN`, `GINE` y `GGNN` forman un segundo bloque de rendimiento alto, mientras que las variantes GraphSAGE, GCN y GAT/GATv2 se sitúan en una zona intermedia. El baseline `MLP+pool` confirma que, en esta tarea, ignorar el intercambio de información entre nodos limita claramente la capacidad predictiva.

8.4. Explicabilidad de las predicciones mediante GNNExplainer

Las métricas cuantitativas permiten comparar el rendimiento de las arquitecturas, pero no explican qué partes de una molécula han sido utilizadas por el modelo para tomar una decisión. Esta cuestión es especialmente relevante en MUTAG, donde cada grafo representa un compuesto químico, los nodos representan átomos y las aristas representan enlaces. Por ello, se incorporó un análisis de explicabilidad post-hoc con el objetivo de estudiar si las predicciones de las GNN se apoyan en subestructuras moleculares localizadas o si, por el contrario, parecen depender de patrones menos interpretables.

Para este análisis se utilizó GNNExplainer, una técnica de explicabilidad específica para redes neuronales sobre grafos. Dado un modelo ya entrenado y una predicción concreta, GNNExplainer busca una subestructura compacta del grafo y un subconjunto de atributos de entrada que preserven la predicción original. De forma intuitiva, el método aprende una máscara sobre aristas y, opcionalmente, una máscara sobre atributos de nodos, de manera que el subgrafo enmascarado conserve la mayor cantidad posible de información relevante para la decisión del modelo.

En el contexto de este trabajo, la explicación se interpreta de la siguiente manera. La máscara de aristas asigna una importancia a cada enlace químico de la molécula, mientras que la máscara de nodos o atributos permite identificar qué átomos han contribuido más a la predicción. Estas explicaciones son locales: no describen el comportamiento global del modelo, sino la decisión tomada para una molécula concreta por un checkpoint concreto.

8.4.1. Protocolo de explicabilidad

Para evitar mezclar entrenamiento y explicabilidad, las explicaciones se calcularon sobre los checkpoints ya seleccionados durante la experimentación principal. En particular, para cada combinación modelo-semilla-pliegue se guardó el checkpoint correspondiente a la mejor época según la métrica de validación interna. Posteriormente, el script de explicabilidad cargó esos pesos sin reentrenar la red.

Para cada modelo se seleccionaron cuatro moléculas representativas:

- una molécula mutagénica correctamente clasificada;
- una molécula mutagénica mal clasificada;
- una molécula no mutagénica correctamente clasificada;
- una molécula no mutagénica mal clasificada.

La selección se hizo agregando las predicciones sobre las distintas semillas. Para los errores se eligieron las moléculas que el modelo fallaba de forma más consistente y, en caso de empate, aquellas para las que el modelo asignaba menor probabilidad media a la clase verdadera. Para los aciertos se eligieron las

moléculas correctamente clasificadas con mayor consistencia y mayor probabilidad media de la clase verdadera.

El resultado de este proceso fue un conjunto de explicaciones locales almacenadas como imágenes y tablas. Cada explicación incluye una visualización del grafo molecular, un ranking de aristas por importancia y un ranking de nodos por importancia.

8.4.2. Ejemplo de explicación en una molécula mutagénica correctamente clasificada

La Figura 11 muestra un ejemplo de explicación para el modelo GINE sobre la molécula 125, clasificada correctamente como mutagénica. En esta visualización, los nodos de mayor tamaño corresponden a átomos con mayor importancia y las aristas más gruesas corresponden a enlaces con mayor peso en la máscara aprendida por GNNExplainer.

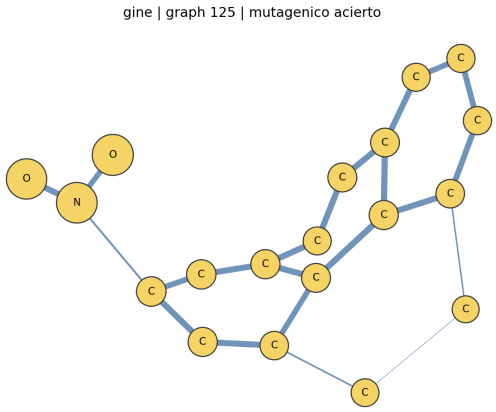

Figura 11: Explicación generada por GNNExplainer para una molécula mutagénica correctamente clasificada por GINE. Elaboración propia.

En este caso, las aristas con mayor importancia corresponden mayoritariamente a enlaces aromáticos, junto con algún enlace doble. Además, los nodos más relevantes incluyen átomos de oxígeno y nitrógeno. Esta combinación resulta interpretable desde el punto de vista del problema, ya que el modelo no parece basarse únicamente en el tamaño del grafo, sino en una subestructura local con información química específica.

La Tabla 4 muestra algunas de las aristas más importantes para esta predicción.

La Tabla 5 recoge los nodos más relevantes. Destaca la presencia de átomos de oxígeno y nitrógeno entre los nodos de mayor importancia.

Cuadro 4: Aristas más importantes para la explicación de GINE en la molécula 125.

Posición	Origen	Destino	Tipo de enlace	Importancia
24	11	2	aromatic	0.888
22	11	10	aromatic	0.885
6	4	3	aromatic	0.871
16	9	8	aromatic	0.868
8	5	4	aromatic	0.866
41	16	17	double	0.856

Cuadro 5: Nodos más importantes para la explicación de GINE en la molécula 125.

Nodo	Tipo	Átomo	Importancia
18	2	O	0.782
16	1	N	0.757
17	2	O	0.742
7	0	C	0.231
6	0	C	0.223

8.4.3. Comparación entre aciertos y errores

Además de estudiar ejemplos individuales, se compararon las explicaciones de aciertos y errores. La Tabla 6 resume algunos casos representativos extraídos de las explicaciones generadas.

Cuadro 6: Ejemplos de moléculas explicadas mediante GNNExplainer.

Modelo	Clase real	Resultado	Molécula	Confianza máx. arista
GINE	mutagénica	acierto	125	0.888
GINE	mutagénica	error	172	0.952
GATv2-edge	mutagénica	acierto	59	0.941
GATv2-edge	mutagénica	error	169	0.941
GINE+GATv2	mutagénica	acierto	92	0.834
GINE+GATv2	mutagénica	error	178	0.938
GAT	no mutagénica	error	78	0.942
GINE	no mutagénica	acierto	186	0.943

En los aciertos mutagénicos se observa con frecuencia una concentración de importancia en enlaces aromáticos y en átomos heteroatómicos como N u O. Por ejemplo, en el caso de GINE sobre la molécula 125, los nodos con mayor importancia son O, N y O, mientras que las aristas más relevantes son principalmente aromáticas. Este comportamiento es coherente con el hecho de que

MUTAG contiene compuestos químicos donde la estructura local y el tipo de enlace pueden ser informativos.

En los errores, las explicaciones sugieren que algunos modelos pueden estar asignando importancia elevada a subestructuras que aparecen también en moléculas mutagénicas, pero que no son suficientes por sí solas para determinar correctamente la clase. Por ejemplo, varias moléculas mutagénicas mal clasificadas aparecen de forma repetida como casos difíciles para distintos modelos, como las moléculas 54, 105, 169, 172 y 178. De forma análoga, la molécula 78 aparece como falso positivo no mutagénico para varios modelos basados en atención.

8.4.4. Patrones agregados observados

Para estudiar patrones de forma más agregada, se analizaron los nodos y aristas con mayor importancia en las explicaciones generadas. En conjunto, los enlaces aromáticos aparecen de forma dominante entre las aristas más relevantes, tanto en aciertos como en errores. Este patrón es esperable en MUTAG, dado el carácter molecular del dataset y la presencia de compuestos aromáticos.

Al separar por tipo de resultado, se observaron las siguientes tendencias:

- En moléculas mutagénicas correctamente clasificadas, las explicaciones suelen destacar enlaces aromáticos y nodos de tipo C, O y N.
- En moléculas mutagénicas mal clasificadas, además de enlaces aromáticos, aparecen con mayor frecuencia enlaces simples y dobles entre las aristas relevantes.
- En algunos errores sobre moléculas no mutagénicas, los modelos asignan alta importancia a subestructuras aromáticas, lo que sugiere que estas pueden actuar como señales ambiguas.
- En los modelos edge-aware, como GINE, GATv2-edge y GINE+GATv2, la máscara de aristas permite observar directamente qué tipos de enlace contribuyen a la decisión.

La Tabla 7 resume el número de apariciones de los tipos de enlace más frecuentes entre las aristas de mayor importancia en las explicaciones.

Estos resultados no deben interpretarse como una regla química causal. Más bien indican qué subestructuras utiliza el modelo para justificar sus decisiones. Por ejemplo, si un modelo asigna sistemáticamente alta importancia a enlaces aromáticos en falsos positivos, esto puede sugerir que está usando la aromaticidad como una señal fuerte de mutagenicidad incluso cuando el contexto molecular completo no justifica esa predicción.

El análisis con GNNExplainer aporta una lectura complementaria a las métricas cuantitativas. Los modelos edge-aware, especialmente GINE y GATv2-edge, permiten obtener explicaciones más ricas porque incorporan explícitamente los atributos de arista. En estos modelos, la explicación no solo indica qué

Cuadro 7: Tipos de enlace más frecuentes entre las aristas más importantes de las explicaciones.

Clase	Resultado	Tipo de enlace	Apariciones
mutagénica	acierto	aromatic	73
mutagénica	error	aromatic	50
mutagénica	error	single	30
no mutagénica	acierto	aromatic	46
no mutagénica	acierto	single	35
no mutagénica	error	aromatic	53
no mutagénica	error	single	20
no mutagénica	error	double	15

átomos son relevantes, sino también qué tipos de enlace contribuyen a la predicción.

En los casos correctamente clasificados como mutagénicos, las explicaciones tienden a resaltar subestructuras aromáticas y heteroátomos. En cambio, algunos errores parecen asociarse a subestructuras que el modelo considera indicativas de mutagenicidad, pero que aparecen también en moléculas no mutagénicas o en contextos moleculares distintos. Esto sugiere que parte de los errores puede deberse a la identificación de patrones locales insuficientemente contextualizados.

Por tanto, la explicabilidad no se utiliza aquí para afirmar que una subestructura sea químicamente causal, sino para analizar el comportamiento aprendido por las redes. Esta distinción es importante: GNNExplainer explica la decisión del modelo, no la propiedad molecular en sí misma.

8.4.5. Limitaciones del análisis XAI

El análisis presenta varias limitaciones. En primer lugar, GNNExplainer produce explicaciones locales, por lo que una explicación individual no resume el comportamiento global de la arquitectura. En segundo lugar, las magnitudes de las máscaras no son directamente comparables entre modelos o moléculas distintas; es más razonable interpretar el ranking relativo de nodos y aristas dentro de cada explicación. En tercer lugar, MUTAG es un dataset pequeño, por lo que los patrones observados deben entenderse como evidencia exploratoria. Finalmente, las explicaciones dependen del checkpoint concreto utilizado, aunque en este trabajo se mitiga parcialmente este problema seleccionando moléculas que son consistentemente acertadas o falladas a través de distintas semillas.

9. Conclusiones

La idea central de este trabajo que atraviesa todo el desarrollo es que el rendimiento de un modelo no depende únicamente de su capacidad expresiva, sino

también de la adecuación entre la arquitectura y la estructura de los datos. En este sentido, el paso de los MLP a las CNN muestra cómo la incorporación de sesgos inductivos como la localidad, la compartición de pesos y la equivarianza permite explotar de forma más eficiente la estructura regular de las imágenes. Las GNN extienden esta misma filosofía a dominios irregulares, sustituyendo la noción de desplazamiento espacial por operaciones de agregación local invariantes a permutaciones.

Las arquitecturas estudiadas pueden interpretarse como distintas instancias de este principio común. GCN introduce una agregación normalizada basada en la estructura del grafo; GraphSAGE formula un enfoque inductivo mediante agregadores de vecindario; GIN se motiva por su capacidad expresiva en relación con el test de Weisfeiler-Lehman; GAT y GATv2 incorporan mecanismos de atención para ponderar la contribución de los vecinos; y GGNN introduce una actualización recurrente mediante compuertas. Esta diversidad muestra que no existe una única forma de generalizar la convolución a grafos, sino una familia de diseños que combinan, con distintos grados de flexibilidad, localidad, compartición de parámetros, agregación invariante y capacidad de incorporar atributos de nodos y aristas.

La experimentación con MUTAG ha permitido conectar estos fundamentos con una tarea concreta de clasificación de grafos moleculares. En este problema, cada molécula se representa como un grafo cuyos nodos son átomos y cuyas aristas son enlaces químicos, por lo que la predicción exige transformar información local en una representación global de la molécula. Los resultados muestran que los modelos con message passing superan claramente al baseline MLP+pool, que no intercambia información entre nodos. Esta diferencia confirma empíricamente la importancia de utilizar arquitecturas alineadas con la estructura relacional del dato: en una molécula no basta con conocer los tipos de átomos de forma aislada, sino que resulta fundamental modelar cómo se conectan entre sí.

Entre los modelos evaluados, el híbrido GINE+GATv2 obtiene el mejor rendimiento medio en las tres métricas consideradas: accuracy, balanced accuracy y F1 macro. Este resultado es coherente con la motivación de la arquitectura, ya que combina una primera etapa capaz de incorporar explícitamente atributos de arista con una etapa atencional que pondera dinámicamente la información procedente de los vecinos. También destacan GIN, GINE y GGNN como modelos de alto rendimiento, lo que sugiere que, en MUTAG, la capacidad de agregación expresiva y la consideración de la estructura local son especialmente relevantes. Además, la mejora de algunas variantes edge-aware frente a sus equivalentes sin atributos de arista refuerza la idea de que el tipo de enlace químico aporta información útil para la clasificación molecular.

Estos resultados también pueden contextualizarse con los reportados en la literatura para MUTAG. En particular, Xu et al. [21]obtienen accuracies superiores para GIN, con valores de $89{,}4 \pm 5{,}6$ para GIN-0 y $89{,}0 \pm 6{,}0$ para GIN-ϵ. En este trabajo, el mejor modelo alcanza una accuracy media de $83{,}92\,\%$. Esta diferencia no debe interpretarse como una comparación directa entre implementaciones, ya que el protocolo experimental no es idéntico. En nuestro caso se emplean varias semillas, validación interna para la selección del modelo, early

stopping y métricas adicionales como balanced accuracy y F1 macro. Por tanto, los resultados de Xu et al. funcionan como una referencia externa de rendimiento, mientras que la comparación principal de este trabajo se basa en el protocolo homogéneo aplicado a todas las arquitecturas evaluadas.

El análisis de explicabilidad mediante GNNExplainer complementa las métricas cuantitativas al mostrar qué partes de las moléculas influyen en determinadas predicciones. Las explicaciones obtenidas indican que, en varios aciertos mutagénicos, los modelos concentran importancia en subestructuras aromáticas y en átomos como oxígeno y nitrógeno. Este comportamiento es interpretable desde el punto de vista del problema, ya que la predicción no parece depender únicamente del tamaño del grafo o de características globales simples, sino de patrones locales con significado químico. Al mismo tiempo, los errores analizados muestran que algunas subestructuras pueden actuar como señales ambiguas: el modelo puede atribuir alta importancia a enlaces aromáticos o a ciertos patrones locales que también aparecen en moléculas no mutagénicas o en contextos donde no son suficientes para justificar la clase correcta.

No obstante, las conclusiones experimentales deben interpretarse con cautela. MUTAG es un dataset pequeño, formado por 188 grafos, por lo que las estimaciones de rendimiento presentan una variabilidad notable entre folds y semillas. Por tanto, el objetivo de la experimentación debe entenderse como una comparación razonada bajo un protocolo homogéneo, no como una búsqueda exhaustiva del máximo rendimiento posible.

También existen limitaciones asociadas a la propia representación molecular y al análisis de explicabilidad. Las moléculas se representan mediante etiquetas discretas de átomos y enlaces codificadas en one-hot, sin incorporar otros descriptores químicos potencialmente relevantes, como cargas parciales, geometría tridimensional, información estereoquímica o propiedades fisicoquímicas. Por otra parte, GNNExplainer proporciona explicaciones locales de predicciones concretas y sus máscaras no son directamente comparables entre modelos o moléculas distintas. Estas explicaciones ayudan a entender el comportamiento aprendido por una red, pero no deben interpretarse como pruebas de causalidad química. En consecuencia, el análisis XAI aporta indicios útiles sobre los patrones utilizados por los modelos, pero no sustituye una validación química externa.

Como líneas futuras, sería conveniente ampliar la evaluación a otros benchmarks moleculares de mayor tamaño, realizar búsquedas sistemáticas de hiperparámetros, incorporar pruebas estadísticas sobre las diferencias de rendimiento y estudiar ablations específicas sobre el uso de atributos de arista, el tipo de readout y el número de capas. También sería interesante comparar las GNN con descriptores moleculares clásicos o fingerprints químicos, así como explorar representaciones más ricas que integren información química adicional. En conjunto, el trabajo muestra que las redes neuronales sobre grafos constituyen una herramienta adecuada para problemas en los que la estructura relacional es parte esencial de la información, pero también pone de manifiesto que su evaluación debe acompañarse de protocolos rigurosos, análisis de estabilidad e interpretaciones prudentes de los resultados.

Referencias

[1] V. N. Vapnik. An overview of statistical learning theory.

[2] S. Shalev-Shwartz and S. Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014.

[3] S. Haykin. *Neural Networks and Learning Machines*. 3rd ed., Pearson, 2009.

[4] G. James, D. Witten, T. Hastie, and R. Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. 2nd ed., Springer, 2021.

[5] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. Adaptive Computation and Machine Learning. MIT Press, Cambridge, MA, 2016.

[6] F. R. K. Chung. *Spectral Graph Theory*. CBMS Regional Conference Series in Mathematics, No. 92, AMS, 1997.

[7] N. J. Nilsson. *Learning Machines: Foundations of Trainable Pattern-Classifying Systems*. McGraw-Hill Series in Systems Science. McGraw-Hill, New York, 1965.

[8] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2009.

[9] M. Defferrard, X. Bresson, and P. Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. (arXiv:1606.09375).

[10] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2:303–314, 1989.

[11] K. Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.

[12] M. Leshno, V. Ya. Lin, A. Pinkus, and S. Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6):861–867, 1993.

[13] R. Balakrishnan and K. Ranganathan. *A Textbook of Graph Theory*. Universitext. Springer, New York, 2nd edition, 2012.

[14] D. I. Shuman, S. K. Narang, P. Frossard, A. Ortega, and P. Vandergheynst. The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains. *IEEE Signal Processing Magazine*, 30(3):83–98, 2013.

[15] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković. Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges. arXiv preprint arXiv:2104.13478, 2021.

[16] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, PMLR 70:1263–1272, 2017.

[17] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip. A comprehensive survey on graph neural networks. *arXiv preprint* arXiv:1901.00596, v4, 2020.

[18] Thomas N. Kipf and Max Welling. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[19] Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper Insights Into Graph Convolutional Networks for Semi-Supervised Learning. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18)*, pages 3538–3545, 2018.

[20] Kenta Oono and Taiji Suzuki. Graph Neural Networks Exponentially Lose Expressive Power for Node Classification. In *International Conference on Learning Representations (ICLR)*, 2020.

[21] K. Xu, W. Hu, J. Leskovec, and S. Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019. (arXiv:1810.00826).

[22] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. Smola. Deep sets. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[23] PyG Team. `torch_geometric.nn.conv.SAGEConv`. *PyTorch Geometric 2.7.0 documentation*, 2025. `https://pytorch-geometric.readthedocs.io/en/2.7.0/generated/torch_geometric.nn.conv.SAGEConv.html` Consultado el 10 de mayo de 2026.

[24] B. Weisfeiler and A. A. Lehman. A reduction of a graph to a canonical form and an algebra arising during this reduction. *Nauchno-Technicheskaya Informatsia*, 2(9):12–16, 1968.

[25] K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[26] K. Xu, C. Li, Y. Tian, T. Sonobe, K.-I. Kawarabayashi, and S. Jegelka. Representation learning on graphs with jumping knowledge networks. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, PMLR 80, 2018. (arXiv:1806.03536).

[27] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec. Strategies for pre-training graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2020. (arXiv:1905.12265).

[28] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec. Open Graph Benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems*, 33, 2020.

[29] C. Morris, N. M. Kriege, F. Bause, K. Kersting, P. Mutzel, and M. Neumann. TUDataset: A collection of benchmark datasets for learning with graphs. In *ICML Workshop on Graph Representation Learning and Beyond*, 2020.

[30] PyTorch Geometric Team. Creating message passing networks. *PyTorch Geometric Documentation*. Disponible en: `https://pytorch-geometric.readthedocs.io/en/latest/tutorial/create\_gnn.html`. (Consultado en 2026).

[31] PyTorch Geometric Team. `torch\_geometric.nn.conv.GINConv`. *PyTorch Geometric Documentation*. Disponible en: `https://pytorch-geometric.readthedocs.io/en/latest/generated/torch\_geometric.nn.conv.GINConv.html`. (Consultado en 2026).

[32] PyG Team. `torch_geometric.nn.conv.GINEConv`. *PyTorch Geometric 2.7.0 documentation*, 2025. `https://pytorch-geometric.readthedocs.io/en/2.7.0/generated/torch_geometric.nn.conv.GINEConv.html` Consultado el 10 de mayo de 2026.

[33] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018. (arXiv:1710.10903).

[34] S. Brody, U. Alon, and E. Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations (ICLR)*, 2022. (arXiv:2105.14491).

[35] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*, 2015. (arXiv:1409.0473).

[36] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[37] PyTorch Geometric Team. `torch\_geometric.nn.conv.GATConv`. *PyTorch Geometric Documentation*. Disponible en: `https://pytorch-geometric.readthedocs.io/en/latest/generated/torch\_geometric.nn.conv.GATConv.html`. (Consultado en 2026).

[38] PyTorch Geometric Team. `torch\_geometric.nn.conv.GATv2Conv`. *PyTorch Geometric Documentation*. Disponible en: `https://pytorch-geometric.readthedocs.io/en/latest/generated/torch\_geometric.nn.conv.GATv2Conv.html`.

[39] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel. Gated graph sequence neural networks. In *International Conference on Learning Representations (ICLR)*, 2016. (arXiv:1511.05493).

[40] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of EMNLP*, pages 1724–1734, 2014. (arXiv:1406.1078).

[41] PyTorch Geometric Team. `torch\_geometric.nn.conv.GatedGraphConv`. *PyTorch Geometric Documentation*. Disponible en: `https://pytorch-geometric.readthedocs.io/en/latest/generated/torch\_geometric.nn.conv.GatedGraphConv.html`. (Consultado en 2026).

[42] M. Fey and J. E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. (arXiv:1903.02428).

[43] A. Daigavane, B. Ravindran, and G. Aggarwal. Understanding Convolutions on Graphs. *Distill*, 2021. doi:10.23915/distill.00032.

[44] B. Sanchez-Lengeling, E. Reif, A. Pearce, and A. B. Wiltschko. A Gentle Introduction to Graph Neural Networks. *Distill*, 2021. doi:10.23915/distill.00033.

[45] P. Khadka. *Understanding Convolutional Neural Network (CNNs)*. Medium, 18 July 2024. Disponible en: `https://medium.com/@pranjalkhadka/understanding-convolutional-neural-network-cnns-e6f6e58fe249`. Consultado el 6 de julio de 2026.

[46] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun. Spectral Networks and Locally Connected Networks on Graphs. *arXiv preprint* arXiv:1312.6203, 2014.

[47] W. L. Hamilton, R. Ying, and J. Leskovec. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. (arXiv:1706.02216).

[48] PyG Team. `torch\_geometric.nn.conv.GCNConv`. *PyTorch Geometric 2.7.0 documentation*, 2025. `https://pytorch-geometric.readthedocs.io/en/2.7.0/generated/torch_geometric.nn.conv.GCNConv.html` Consultado el 10 de mayo de 2026.

[49] P. W. Battaglia, J. B. Hamrick, V. Bapst, *et al.* Relational inductive biases, deep learning, and graph networks. *arXiv preprint* arXiv:1806.01261, 2018.